# Unidade 2

## Apresentação

Os bancos de dados são utilizados diariamente em diferentes contextos e, à medida que o volume de informações aumenta, torna-se fundamental realizar consultas de forma rápida, organizada e eficiente. Para isso, a linguagem SQL oferece diversos recursos para recuperação, organização e análise dos dados.

Entre esses recursos estão os índices, estruturas utilizadas para otimizar a localização e a recuperação de registros em bancos de dados, além das funções de agregação e agrupamento, que permitem resumir, organizar e analisar conjuntos de informações. A SQL também possibilita a construção de consultas mais complexas, utilizando recursos como junções, ordenação, aninhamento e correlacionamento.

Nesta Unidade de Aprendizagem, você compreenderá a importância e a aplicação dos índices em bancos de dados, conhecerá as principais funções de agregação e agrupamento em SQL e aprenderá a desenvolver consultas avançadas para recuperar e analisar informações de maneira eficiente.

Bons estudos.

### Ao final desta Unidade de Aprendizagem, você deve apresentar os seguintes aprendizados:

- Definir o conceito de índices em banco de dados e identificar suas principais vantagens.
- Criar, excluir e aplicar índices em consultas SQL.
- Identificar e utilizar funções de agregação e agrupamento de dados em SQL.
- Desenvolver consultas utilizando junção, agregação e ordenação.
- Produzir consultas avançadas empregando aninhamento e correlacionamento.

# Infográfico: Funções de agregação e agrupamento de dados em SQL

As funções de agrupamento selecionam um conjunto de linhas para uso das funções de agregação. As linhas são associadas a grupos definidos por um atributo que faça parte da consulta. Dessa forma, a cada grupo são executadas as funções de agregação e apresentadas as informações associadas a cada um deles.

Neste Infográfico, você verá os passos de execução da função de agrupamento GROUP BY com a função de agrupamento HAVING, associada à função de agregação.

# FUNÇÕES DE AGRUPAMENTO

As funções de agrupamento são executadas para selecionar um grupo de linhas. Este pode ser formado a partir de atributos que podem ter conteúdo igual. Essas funções são a **GROUP BY e HAVING**.

## GROUP BY

As funções de agrupamento são combinadas com as funções de agregação: **count, min, max, sum e avg**. Assim, é extraída a informação do grupo.

**Observe:**



| ATLETAS | | | | |
|---|---|---|---|---|
| CODIGO | NOME | PESO | ALTURA | ESPORTE |
| 10 | GABRIEL | 80 | 1,79 | VOLEI |
| 20 | MARIANA | 57 | 1,67 | FUTEBOL |
| 30 | JULIA | 80 | 1,70 | FUTEBOL |

Pode-se agrupar por: PESO, ALTURA, ESPORTE

**Qual é a quantidade de atletas por esporte?**

O agrupamento vai visualizar as informações em grupo. O processamento da consulta irá contar quantos atletas praticam cada esporte armazenado. Para contar, a SQL disponibiliza a função de agregação COUNT e, para agrupar a função de agrupamento, o GROUP BY.

```
Select count (*) from atletas group by esporte;
```

| ESPORTE | Count (*) |
|---|---|
| VOLEI | 1 |
| FUTEBOL | 2 |

## HAVING32

Aplica-se a função HAVING nas linhas resultantes da execução da função de agrupamento GROUP BY.

O agrupamento vai visualizar as informações em grupo, mas o que se pede aqui é mostrar somente o esporte que tenha **mais de 1 atleta praticando**. Dessa forma, é preciso testar o resultado do agrupamento e apresentar somente as linhas que atendem à condição. O comando da SQL utilizado para testar o resultado do GROUP BY é o HAVING.

```
Select count (*) from atletas
                group by esporte
              having count (*) > 1;
```

| ESPORTE | Count (*) |
|---|---|
| ~~VOLEI~~ | ~~1~~ |
| FUTEBOL | 2 |

| ESPORTE | Count (*) |
|---|---|
| FUTEBOL | 2 |

Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

# Conteúdo do Livro: Funções de agregação e agrupamento de dados em SQL

Os Sistemas Gerenciadores de Bancos de Dados Relacionais utilizam a SQL para definição, manipulação e consulta a bancos de dados relacionais. Dentre as diversas instruções definidas na SQL, as funções de agregação, combinadas com as funções de agrupamento, apresentam-se para formular consultas que requerem a análise em grupos de linhas selecionadas ao invés de linhas individualizadas.

Considera-se como função de agregação o COUNT, o SUM, o AVG, o MAX e o MIN. O COUNT é a função que apresenta o quantitativo de linhas resultado da consulta. O SUM mostra a soma de uma coluna que armazena valores numéricos de qualquer dimensão. O AVG calcula a média da coluna indicada. O MIN verifica o valor mínimo armazenado na coluna e, o MAX, o valor máximo.

As funções de agrupamento são representadas pelas instruções GROUP BY e HAVING. Estas funções apresentam uma importância relevante na realização das consultas, pois, por meio delas, é possível analisar grupos de informações selecionadas a partir de uma determinada condição.

No capítulo Funções de agregação e agrupamento de dados em SQL, da obra *Consultas em Banco de Dados*, você vai conhecer as funções de agregação e as de agrupamento, além de várias formas de implementação com um caso de estudo.

Boa leitura.

# CONSULTAS EM BANCO DE DADOS

**Claudia Abreu Paes**



SAGAH
SOLUÇÕES EDUCACIONAIS INTEGRADAS

# Funções de agregação e agrupamento de dados em SQL

## Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Identificar as principais funções de agregação em SQL.
- Descrever o agrupamento de dados em SQL.
- Implementar as funções de agregação e agrupamento de dados em SQL.

## Introdução

O uso do banco de dados tem sido cada vez mais difundido nas operações diárias de qualquer segmento de mercado (comércio, indústria, saúde, educação), assim como nos relacionamentos sociais, oferecendo suporte aos procedimentos realizados pelas empresas para os três níveis organizacionais — operacional, tácito e estratégico —, que utilizam os dados de acordo com sua função e proposta no contexto da empresa.

A linguagem SQL (Structured Query Language) dispõe de instruções de definição de dados (DDL), manipulação de dados (DML), consulta de dados (DQL), controle de dados (DCL) e de controle de transações (TCL).

Neste capítulo, você conhecerá o conjunto de instruções com a função de agregação e agrupamento de dados, apresentadas pelas instruções SQL do subtipo DQL (Data Query Language).

# 1 Principais funções de agregação em SQL

Segundo o dicionário *on-line* Dicio, **agregação** significa associação, ou seja, um conjunto de partes homogêneas formando um todo (AGREGAÇÃO, 2020). Por essa definição, consegue-se entender que a função de agregação consiste em realizar o procedimento a todas as ocorrências da informação definida para consulta.

O *site* oficial do *software* PostgreSQL, em sua documentação, define função de agregação como um procedimento que calcula um único valor de resultado de um conjunto de valores de entrada (AGGREGATE..., 2020).

Segundo Silberschatz, Korth e Sudarshan (1999), uma função de agregação pode ser usada nos mesmos lugares em que se usa uma constante, dando-se na declaração do atributo na sintaxe do comando `SELECT`.

```
SELECT <função de agregação> FROM nomeTabela;
```

O `SELECT` com as funções agregadas poderá ser usado incluindo também a cláusula `WHERE` da forma necessária. Assim:

```
SELECT <função de agregação> FROM nomeTabela
    WHERE .........;
```

A seguir, estudaremos as funções de agregação `COUNT`, `SUM`, `AVG`, `MAX` e `MIN` que, segundo Mannino (2008), são as funções de agregação-padrão.

## Função de agregação `COUNT`

Utilizada para contar o número de linhas obtidas no resultado da seleção, seu formato de utilização é:

```
SELECT COUNT (parâmetro) FROM nomeTabela;
```

onde `parâmetro` pode ser `*`, representando todas as linhas, ou `nomeColuna`, especificando a contagem por uma coluna específica.



## Fique atento

Ao usarmos o asterisco (`*`), o resultado considerará todas as linhas, enquanto, ao especificarmos uma coluna por meio de (`nomeColuna`), somente as linhas que tiverem valores válidos (não nulos) serão incluídas no resultado.

Como exemplo, consideraremos para todas as funções de agregação a seguinte tabela, chamada `paciente`:

| CPF | NOME | CIDADE | SEXO | IDADE |
|---|---|---|---|---|
| 11111111111 | JULIA SANTOS | RIO DE JANEIRO | F | 35 |
| 22222222222 | MARIA OLIVEIRA | null | F | 30 |
| 33333333333 | JOÃO DA SILVA | null | M | 25 |
| 44444444444 | CARLOS FERREIRA | NITERÓI | M | 28 |

Utilizando a instrução de consulta:

```
SELECT COUNT(*) FROM paciente;
```

O resultado será:

| COUNT(*) |
|---|
| 4 |

Empregando a instrução de consulta:

```
SELECT COUNT(cidade) FROM paciente;
```

O resultado será:

| COUNT(*) |
|---|
| 2 |

As informações de `CIDADE` para os pacientes `JOÃO DA SILVA` e `MARIA OLIVEIRA` estão com valor `null`. Assim, não foram consideradas na contagem.



**Saiba mais**

O valor `null` na linguagem SQL significa vazio. Não é zero e não é branco: é sem valor, não preenchido.

## Função de agregação `SUM`

Empregada para somar os valores armazenados na coluna referente à informação desejada, especificada como parâmetro no operador, por meio do nome do atributo, seu formato de utilização consiste em:

```
SELECT SUM (nomeAtributo) FROM nomeTabela;
```



**Saiba mais**

A coluna referente ao atributo para utilização na função de agregação `SUM` deverá ser do tipo numérico, inteiro ou decimal, sem distinção.



**Exemplo**

Como exemplo, considere mais uma vez os registros da tabela `paciente`:
Utilizando a instrução de consulta:

```
SELECT SUM (idade) FROM paciente;
```

O resultado será:

| SUM(idade) |
|---|
| 118 |

## Função de agregação AVG

Utilizada para calcular a média dos valores armazenados na coluna, especificada como parâmetro na instrução, pelo nome do atributo, seu formato de utilização é:

```
SELECT AVG (nomeAtributo) FROM nomeTabela;
```



### Saiba mais

A coluna referente ao atributo para utilização na função de agregação AVG deverá ser do tipo numérico, inteiro ou decimal, sem distinção.



### Exemplo

Como exemplo, considere novamente a tabela paciente do Quadro 1.

Utilizando a instrução de consulta:

```
SELECT AVG (idade) FROM paciente;
```

O resultado será:

| AVG (idade) |
|---|
| 29,50 |



### Saiba mais

A palavra-chave DISTINCT pode ser utilizada junto às funções de agregação COUNT, AVG e SUM para considerar valores únicos, sem repetição, em determinada coluna.

## Função de agregação `MIN`

Empregada para apresentar o **menor valor** armazenado na coluna especificada como parâmetro na instrução, por meio do nome do atributo, seu formato de utilização consiste em:

```
SELECT MIN (nomeAtributo) FROM nomeTabela;
```



### Exemplo

Como exemplo, considere novamente a tabela `paciente`:
Utilizando a instrução de consulta:

```
SELECT MIN (idade) FROM paciente;
```

O resultado será:

| MIN (idade) |
|---|
| 25 |

## Função de agregação `MAX`

Já a função de agregação `MAX` é utilizada para apresentar o **maior valor** armazenado na coluna especificada como parâmetro na instrução, por meio do nome do atributo. Seu formato de utilização é:

```
SELECT MAX (nomeAtributo) FROM nomeTabela;
```



### Exemplo

Como exemplo, tenha mais uma vez em conta os dados da tabela `paciente`:
Utilizando a instrução de consulta:

```
SELECT MAX (idade) FROM paciente;
```

O resultado será:

| MAX(idade) |
|---|
| 35 |



### Saiba mais

No uso das funções de agregação nas consultas SQL, o nome da coluna apresentada refere-se à função utilizada. Pode-se adotar o termo `AS` para atribuir um "apelido" à coluna e, assim, melhorar a visibilidade:

```
SELECT MAX (idade) AS MaiorIdade FROM paciente;
```

O resultado será:

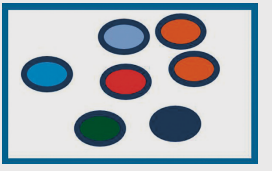
| MaiorIdade |
|---|
| 35 |

## 2 Agrupamento de dados em SQL

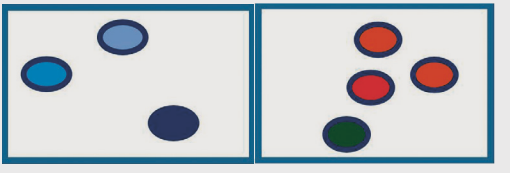
Quando falamos em agrupamento, estamos nos referindo a ajuntamento, não é mesmo? Assim, podemos imaginar que, de um conjunto de bolas com tons de cores diferentes, formaremos ajuntamentos sob um critério definido.

**Exemplo**

Dado o seguinte conjunto:



Desse conjunto, desejamos formar dois ajuntamentos: um de bolas no tom AZUL e outro com bolas de tons diferentes de AZUL. Teremos assim:



Analisando os ajuntamentos, podemos dizer que o ajuntamento das bolas no tom AZUL tem 3 elementos e o das bolas no tom diferente de AZUL apresenta 4 elementos.

O agrupamento de dados acontece da seguinte maneira: podemos juntar um conjunto de linhas associado a um critério, formando vários grupos que se encaixam no critério estabelecido.

O *site* oficial da Microsoft (SELECT..., 2019), em sua documentação, define que a execução de um agrupamento de dados dividirá o resultado e gerará uma linha para cada grupo.

Mannino (2008) apresenta a função de agrupamento pelas cláusulas `GROUP BY` e `HAVING`, relevantes na realização das consultas, já que por meio delas é possível analisar grupos de informações selecionadas a partir de determinada condição, como verificar informações de vendas por vendedor.

A cláusula `GROUP BY` é aplicada ao conjunto de dados, formando ajuntamento de linhas para execução das funções agregadas; já a cláusula `HAVING`, associada ao `GROUP BY`, realizará uma condição de seleção dos dados resultantes.

## Função de agrupamento — cláusula `GROUP BY`

Segundo Silberschatz, Korth e Sudarshan (1999), uma cláusula `GROUP BY` faz parte da própria consulta, resultando em um agrupamento único de tuplas, independentemente da função de agregação da consulta.

Para execução da função de agrupamento, na cláusula `GROUP BY`, é preciso especificar o atributo que se deseja agrupar. A função de agregação será aplicada ao conjunto de linhas referente a cada grupo.

O formato de utilização da cláusula `GROUP BY` na função de agrupamento é:

```
SELECT nomeAtributo, <função de agregação> FROM nomeTabela
GROUP BY nomeAtributo;
```



### Exemplo

Como exemplo, considere agora a tabela `estudante`.

| CPF | NOME | CIDADE | SEXO | IDADE |
|---|---|---|---|---|
| 55555555555 | GABRIELA ALVES | RIO DE JANEIRO | F | 22 |
| 66666666666 | HELENA LIMA | RIO DE JANEIRO | F | 28 |
| 77777777777 | ANDRÉ TORRES | RIO DE JANEIRO | M | 32 |
| 88888888888 | MIGUEL FERREIRA | NITERÓI | M | 26 |

Utilizando a instrução de consulta para conhecer a quantidade de estudantes por cidade, teremos:

```
SELECT cidade, COUNT (*) FROM estudante
    GROUP BY cidade;
```

Analisando a execução da cláusula `GROUP BY` na tabela `estudante`, podemos identificar dois grupos: RIO DE JANEIRO e NITERÓI. A função de agregação utilizada foi `COUNT`. Assim, deverá contar o número de linhas para cada grupo identificado. Para o grupo da cidade RIO DE JANEIRO, conseguimos contar 3 linhas, ou 3 estudantes, e, para o da cidade NITERÓI, 1 linha, ou 1 estudante.

Então, o resultado da consulta será:

| CIDADE | COUNT(*) |
|---|---|
| RIO DE JANEIRO | 3 |
| NITERÓI | 1 |

O processamento da cláusula `GROUP BY` agrupa as linhas de mesmo conteúdo do atributo/coluna que se deseja agrupar e, em seguida, realiza a operação comandada na lista de atributos. Nesse contexto, é possível utilizar todas as funções agregadas (`COUNT`, `SUM`, `MAX`, `MIN` e `AVG`).

A cláusula `GROUP BY` da função de agrupamento poderá ser usada com a cláusula `WHERE` nas consultas. A execução do agrupamento se dará em cima das linhas geradas como resultado da seleção. Assim:

```
SELECT nomeAtributo, <função de agregação> FROM nomeTabela
WHERE .........;
GROUP BY nomeAtributo;
```



### Exemplo

Como exemplo, considere novamente a tabela `estudante`.

Utilizando a instrução de consulta para conhecer a quantidade de estudantes por cidade, com idade inferior a 30 anos, teremos:

```
SELECT cidade, COUNT (*) FROM estudante
    WHERE idade < 30
    GROUP BY cidade;
```

Na tabela `estudante`, podemos identificar os grupos RIO DE JANEIRO e NITERÓI, mas devemos desconsiderar a linha correspondente a pacientes com idade igual ou acima de 30 anos, pois não atendem à condição apresentada na cláusula `WHERE`. Assim, teríamos como resultado da consulta as seguintes linhas, em memória:

| CPF | NOME | CIDADE | SEXO | IDADE |
|---|---|---|---|---|
| 55555555555 | GABRIELA ALVES | RIO DE JANEIRO | F | 22 |
| 66666666666 | HELENA LIMA | RIO DE JANEIRO | F | 28 |
| 88888888888 | MIGUEL FERREIRA | NITERÓI | M | 26 |

O próximo passo consiste em aplicar a função de agregação `COUNT`, que contará o número de linhas para cada grupo identificado nesse conjunto de dados. Para o grupo da cidade RIO DE JANEIRO, conseguimos contar 2 linhas, ou 2 pacientes com idade inferior a 30 anos, e, para o da cidade NITERÓI, 1 linha, ou 1 paciente com idade inferior a 30 anos.

Então, o resultado da consulta será:

| CIDADE | COUNT(*) |
|---|---|
| RIO DE JANEIRO | 2 |
| NITERÓI | 1 |



**Fique atento**

O nome do atributo utilizado na projeção da instrução `SELECT` deverá estar representado junto ao `GROUP BY`.

Por exemplo:

```
SELECT cidade, AVG(idade) FROM estudante
GROUP BY cidade;
```

## Função de agrupamento — cláusula `HAVING`

A cláusula `HAVING` da função de agrupamento tem como objetivo selecionar linhas resultantes da execução do `GROUP BY`, cujo processamento acontece como uma subconsulta.

Seu formato de utilização consiste em:

```
SELECT nomeAtributo, funçãoDeAgregação FROM nomeTabela
   GROUP BY nomeAtributo
       HAVING condição;
```



## Exemplo

Tenha em conta novamente a tabela `estudante`.

Suponha que se deseja conhecer a quantidade de estudantes por cidade, mas apenas interessam as cidades que tenham acima de um estudante.

Como ficaria a instrução?

```
SELECT cidade, COUNT (*) FROM estudante
    GROUP BY cidade
    HAVING COUNT(*) > 1;
```

Na tabela `estudante`, podemos identificar dois grupos: RIO DE JANEIRO e NITERÓI:

A função de agregação utilizada foi `COUNT`. Assim, deverá contar o número de linhas para cada grupo identificado. Para o grupo da cidade RIO DE JANEIRO, conseguimos contar 3 linhas, ou 3 pacientes, e, para o da cidade NITERÓI, 1 linha, ou 1 paciente.

No resultado da função de agrupamento `GROUP BY`, executamos a função de agrupamento `HAVING`, pois só interessa saber a quantidade de pacientes para cidades com mais de 1 paciente, concorda?

Desse modo, devemos desconsiderar a informação da cidade de NITERÓI para o resultado final da consulta. Então, o resultado final da consulta será:

| CIDADE | COUNT(*) |
|---|---|
| RIO DE JANEIRO | 3 |



## Fique atento

Os resultados parciais gerados pela execução da consulta não aparecem para o usuário. O processamento é interno; e o armazenamento, em memória.

# 3 Implementação das funções de agregação e agrupamento de dados

Para demonstrar formas de implementação das funções de agregação e agrupamento, utilizaremos a situação de negócio descrita a seguir.

A (hipotética) Clínica Médica Felizes por Viver é uma clínica geriátrica cujo quadro funcional dispõe de vários médicos para atendimento. Funcionando 7 dias/semana × 24 horas/dia, realiza atendimentos de emergência e rotina. Na entrada do paciente na clínica, que deve ser registrado, registra-se uma consulta, que também pode ser agendada. Os médicos registram os atendimentos, classificando-os pelos procedimentos realizados na consulta.

Os dados a serem utilizados pela Clínica Médica Felizes por Viver apresentam-se modelados em entidades e relacionamentos, representados na Figura 1.

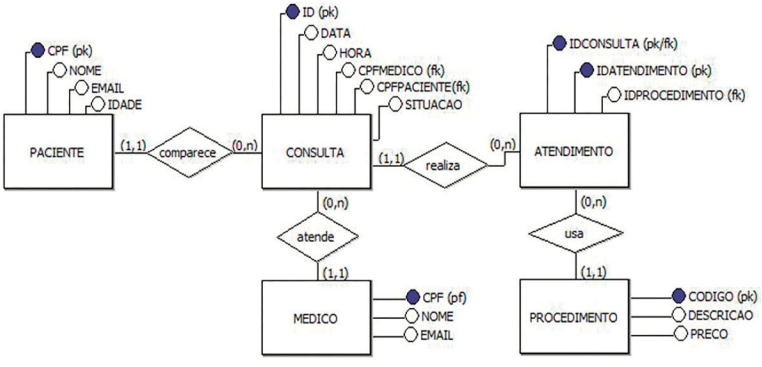



**Figura 1.** Modelo de entidade e relacionamento.

Para aplicação das consultas e da geração dos resultados, serão utilizadas as seguintes informações, a título de exemplo, para cada tabela do modelo.

`procedimento`

| CODIGO (PK) | DESCRICAO | PRECO |
|---|---|---|
| 1 | CONSULTA | 150,00 |
| 2 | APLICAÇÃO INJEÇÃO | 60,00 |
| 3 | CURATIVO | 40,00 |
| 4 | REVISAO EXAMES | 80,00 |

`paciente`

| CPF (PK) | NOME | EMAIL | IDADE |
|---|---|---|---|
| 10101010101 | ANTONIO MANOEL | amanoel@fviver.com.br | 67 |
| 20202020202 | MARIA JOAQUINA | mjoaquina@fviver.com.br | 70 |
| 30303030303 | JULIA SILVA | jsilva@fviver.com.br | 80 |

`medico`

| CRM (PK) | NOME | EMAIL |
|---|---|---|
| 567890 | MARCELO ARAÚJO | maraujo@fviver.com.br |
| 234891 | ALEXANDRE SOUZA | asouza@fviver.com.br |
| 167384 | JUNIA TRINDADE | jtrindade@fviver.com.br |

consulta

| ID (PK) | DATA | HORA | CRMMEDICO (FK) | CPFPACIENTE (FK) | SITUACAO |
|---|---|---|---|---|---|
| 1 | 10/01/2020 | 8:00 | 567890 | 10101010101 | realizada |
| 2 | 10/01/2020 | 10:00 | 234891 | 30303030303 | realizada |
| 3 | 15/01/2020 | 8:00 | 567890 | 30303030303 | realizada |
| 4 | 12/02/2020 | 14:00 | 567890 | 10101010101 | realizada |
| 5 | 20/02/2020 | 16:00 | 234891 | 20202020202 | realizada |
| 6 | 10/03/2020 | 9:00 | 167384 | 30303030303 | realizada |
| 7 | 25/03/2020 | 19:00 | | 20202020202 | agendada |
| 8 | 27/03/2020 | 9:00 | | 10101010101 | cancelada |

OBS: situacao = {agendada – realizada – cancelada}

atendimento

| IDCONSULTA (PK/FK) | IDATENDIMENTO (PK) | IDPROCEDIMENTO (FK) |
|---|---|---|
| 1 | 1 | 1 |
| 1 | 2 | 2 |
| 2 | 1 | 2 |
| 3 | 1 | 3 |
| 3 | 2 | 2 |
| 4 | 1 | 4 |
| 5 | 1 | 1 |
| 6 | 1 | 2 |

## Consultas com função de agregação

### Consultas em uma única tabela

Para utilizarmos a função de agregação a uma única tabela, devemos escolher a função e o atributo a serem utilizados no atendimento da necessidade apresentada.



**Exemplo**

Empregando a tabela `paciente`, desejam-se:

- conhecer a quantidade de pacientes da Clínica Médica Felizes por Viver.
  Para atender a essa necessidade, utiliza-se a função de agregação `COUNT`:

```
SELECT COUNT(*) FROM paciente;
```

O resultado será:

| COUNT(*) |
|---|
| 3 |

- conhecer o paciente com a maior idade atendida pela Clínica Médica Felizes por Viver.
  Para atender a essa necessidade, utiliza-se a função de agregação `MAX`:

```
SELECT MAX(idade) FROM paciente;
```

O resultado será:

| MAX(idade) |
|---|
| 80 |

## Consultas a várias tabelas

Implica identificar os atributos necessários e as tabelas correspondentes; em seguida, deve-se igualar chaves primárias com chaves estrangeiras para fazer a junção das tabelas envolvidas na consulta.



### Exemplo

Suponha-se que se deseja conhecer a média de idade dos pacientes atendidos pelo médico ALEXANDRE SOUZA da Clínica Médica Felizes por Viver.

Na construção da consulta, utilizamos a função de agregação `AVG` na tabela `paciente`, pois nela se encontram os dados de `idade`, `consulta` e, também, `medico`, visto que a seleção se dá por `nome` e a informação que se tem em `consulta` é `CPF` para `paciente` e `CRM` para `medico`. Então, deve-se fazer a junção!

```
SELECT AVG(idade) FROM PACIENTE p, MEDICO m, CONSULTA c
WHERE p.cpf = c.cpf and m.cpf = c.cpf and m.nome = "ALE-
XANDRE SOUZA";
```

As linhas selecionadas para execução da função de agregação serão:

`paciente`

| CPF (PK) | NOME | EMAIL | IDADE |
|---|---|---|---|
| 20202020202 | MARIA JOAQUINA | mjoaquina@fviver.com.br | 70 |
| 30303030303 | JULIA SILVA | jsilva@fviver.com.br | 80 |

`medico`

| CRM (PK) | NOME | EMAIL |
|---|---|---|
| 234891 | ALEXANDRE SOUZA | asouza@fviver.com.br |

consulta

| ID (PK) | DATA | HORA | CRMMEDICO (FK) | CPF-PACIENTE (FK) | SITUA-CAO |
|---|---|---|---|---|---|
| 2 | 10/01/2020 | 10:00 | 234891 | 30303030303 | realizada |
| 5 | 20/02/2020 | 16:00 | 234891 | 20202020202 | realizada |

Aplicando a função de agregação `AVG(IDADE)`, tem-se o resultado:

| AVG(idade) |
|---|
| 75 |

Já no caso de se desejar conhecer o faturamento total da Clínica Médica Felizes por Viver no mês de janeiro/2020.

Na construção da consulta, deve-se utilizar a função de agregação `SUM` para acumular o preço cobrado aos procedimentos utilizados nas consultas realizadas pelos pacientes, registrado em `atendimento`. Então, faz-se a junção entre `procedimento` e `consulta`!

```
SELECT SUM(preco) FROM procedimento p, consulta c, aten-
dimento a
     WHERE a.idconsulta = c.id and p.codigo = a.idProcedimento
   and data >= "01/01/2020" and data <= "31/01/2020";
```

As linhas selecionadas para execução da função de agregação serão:

procedimento

| CODIGO (PK) | DESCRICAO | PRECO |
|---|---|---|
| 1 | CONSULTA | 150,00 |
| 2 | APLICAÇÃO INJEÇÃO | 60,00 |
| 3 | CURATIVO | 40,00 |

consulta

| ID (PK) | DATA | HORA | CRMMEDICO (FK) | CPFPACIENTE (FK) | SITUACAO |
|---|---|---|---|---|---|
| 1 | 10/01/2020 | 8:00 | 567890 | 10101010101 | realizada |
| 2 | 10/01/2020 | 10:00 | 234891 | 30303030303 | realizada |
| 3 | 15/01/2020 | 8:00 | 567890 | 30303030303 | realizada |

OBS.: SITUAÇÃO = {agendada – realizada – cancelada}

atendimento

| IDCONSULTA (PK/FK) | IDATENDIMENTO (PK) | IDPROCEDIMENTO |
|---|---|---|
| 1 | 1 | 1 |
| 1 | 2 | 2 |
| 2 | 1 | 2 |
| 3 | 1 | 3 |
| 3 | 2 | 2 |

A cada `idProcedimento` de atendimento, será somado o preço correspondente da tabela `procedimento`. Assim: 150,00 + 60,00 + 60,00 + 40,00 + 60,00 = 370,00.

Portanto, aplicando a função de agregação `SUM (PRECO)`, tem-se o resultado:

| SUM ( preco) |
|---|
| 370,00 |

## Consultas com função de agrupamento

### Consultas em uma única tabela

Para utilização da função de agrupamento a uma única tabela, devemos escolher a função e o atributo de agregação, assim como o atributo a ser agrupado para atender à necessidade apresentada.



## Exemplo

Precisamos conhecer:

- a quantidade de consultas por `situacao` da Clínica Médica Felizes por Viver.

Para atender a essa necessidade, deve-se utilizar a função de agregação `COUNT`, agrupando pelo atributo `situacao`.

```
SELECT situacao, COUNT(*) FROM consulta
   GROUP BY situacao;
```

As linhas selecionadas para execução da função de agregação serão:

| ID (PK) | DATA | HORA | CRMME-DICO (FK) | CPFPACIEN-TE (FK) | SITUACAO |
|---|---|---|---|---|---|
| 7 | 25/03/2020 | 19:00 | | 20202020202 | agendada |
| 8 | 27/03/2020 | 9:00 | | 10101010101 | cancelada |
| 2 | 10/01/2020 | 10:00 | 234891 | 30303030303 | realizada |
| 1 | 10/01/2020 | 8:00 | 567890 | 10101010101 | realizada |
| 3 | 15/01/2020 | 8:00 | 567890 | 30303030303 | realizada |
| 4 | 12/02/2020 | 14:00 | 567890 | 10101010101 | realizada |
| 5 | 20/02/2020 | 16:00 | 234891 | 20202020202 | realizada |
| 6 | 10/03/2020 | 9:00 | 167384 | 30303030303 | realizada |

Aplicando a função de agregação `COUNT(*)`, com agrupamento (`GROUP BY`) de `situacao`, tem-se o resultado:

| SITUACAO | COUNT(*) |
|---|---|
| agendada | 1 |
| realizada | 6 |
| cancelada | 1 |

- a quantidade de consultas realizadas, por `crmmedico`, na Clínica Médica Felizes por Viver.

Para atender a essa necessidade, deve-se utilizar a função de agregação `COUNT`, selecionar somente as consultas realizadas e o atributo `crmmedico` para o agrupamento:

```
SELECT crmmedico, COUNT(*) FROM consulta
  WHERE situacao = "realizada"
       GROUP BY crmmedico;
```

As linhas selecionadas para execução da função de agregação serão:

| ID (PK) | DATA | HORA | CRMMEDICO (FK) | CPFPACIENTE (FK) | SITUACAO |
|---|---|---|---|---|---|
| 2 | 10/01/2020 | 10:00 | 234891 | 30303030303 | realizada |
| 1 | 10/01/2020 | 8:00 | 567890 | 10101010101 | realizada |
| 3 | 15/01/2020 | 8:00 | 567890 | 30303030303 | realizada |
| 4 | 12/02/2020 | 14:00 | 567890 | 10101010101 | realizada |
| 5 | 20/02/2020 | 16:00 | 234891 | 20202020202 | realizada |
| 6 | 10/03/2020 | 9:00 | 167384 | 30303030303 | realizada |

Aplicando a função de agregação `COUNT(*)`, com agrupamento (`GROUP BY`) do `crmmedico` e seleção das consultas realizadas, tem-se o resultado:

| CRMMEDICO (FK) | COUNT(*) |
|---|---|
| 567890 | 3 |
| 234891 | 2 |
| 167384 | 1 |

## Consultas a várias tabelas

Implica identificar os atributos necessários e as tabelas correspondentes assim como na função de agregação. Em seguida, igualam-se as chaves primárias às chaves estrangeiras para fazer a junção das tabelas envolvidas na consulta.



## Exemplo

Nesse caso, precisamos conhecer:

- o paciente com menor idade, atendido por médico na Clínica Médica Felizes por Viver.

Na construção da consulta, deve-se utilizar a função de agregação `MIN` na tabela `paciente`, pois nela se encontram a `idade` e a `consulta` para saber o `paciente` atendido e também o `medico`, pois o agrupamento deve se dar pelo nome do médico. Além disso, é preciso utilizar a seleção de somente consultas realizadas. Então, deve-se fazer a junção!

```
SELECT m.nome, MIN(idade) FROM paciente p, medico m,
consulta c
    WHERE p.cpf = c.cpfpaciente and m.crm = c.crmmedico
   GROUP BY m.nome;
```

As linhas selecionadas para execução da função de agregação serão:

`paciente`

| CPF (PK) | NOME | EMAIL | IDADE |
|---|---|---|---|
| 10101010101 | ANTONIO MANOEL | amanoel@fviver.com.br | 67 |
| 20202020202 | MARIA JOAQUINA | mjoaquina@fviver.com.br | 70 |
| 30303030303 | JULIA SILVA | jsilva@fviver.com.br | 80 |

`medico`

| CRM (PK) | NOME | EMAIL |
|---|---|---|
| 567890 | MARCELO ARAÚJO | maraujo@fviver.com.br |
| 234891 | ALEXANDRE SOUZA | asouza@fviver.com.br |
| 167384 | JUNIA TRINDADE | jtrindade@fviver.com.br |

consulta

| ID (PK) | DATA | HORA | CRMME-DICO (FK) | CPFPACIEN-TE (FK) | SITUACAO |
|---|---|---|---|---|---|
| 1 | 10/01/2020 | 8:00 | 567890 | 10101010101 | realizada |
| 2 | 10/01/2020 | 10:00 | 234891 | 30303030303 | realizada |
| 3 | 15/01/2020 | 8:00 | 567890 | 30303030303 | realizada |
| 4 | 12/02/2020 | 14:00 | 567890 | 10101010101 | realizada |
| 5 | 20/02/2020 | 16:00 | 234891 | 20202020202 | realizada |
| 6 | 10/03/2020 | 9:00 | 167384 | 30303030303 | realizada |

Aplicando a função de agregação `MIN(idade)`, por nome do `medico`, tem-se o resultado:

| NOME | MIN(idade) |
|---|---|
| MARCELO ARAÚJO | 67 |
| ALEXANDRE SOUZA | 70 |
| JUNIA TRINDADE | 80 |

- o nome do médico que realizou mais de um procedimento em seus atendimentos na Clínica Médica Felizes por Viver.

Na construção da consulta, deve-se utilizar a função de agregação `COUNT` para contar quantos procedimentos foram utilizados para `atendimento` das `consultas`, por cada `medico`, além da função de agrupamento `HAVING` para selecionar, após o agrupamento, somente os médicos que tenham empregado mais de um procedimento nos atendimentos das consultas realizadas. Então, deve-se fazer a junção entre `procedimento`, `consulta` e `atendimento`!

```
SELECT m.nome, COUNT(*) FROM procedimento p, consulta c,
atendimento a
     WHERE a.idconsulta = c.id and p.codigo = a.idProcedimento
   GROUP BY m.nome,
   HAVING COUNT(*) > 1;
```

As linhas selecionadas para execução da função de agregação, a partir da junção, serão:

`procedimento`

| CODIGO (PK) | DESCRICAO | PRECO |
|---|---|---|
| 1 | CONSULTA | 150,00 |
| 2 | APLICAÇÃO INJEÇÃO | 60,00 |
| 3 | CURATIVO | 40,00 |
| 4 | REVISAO EXAMES | 80,00 |

`medico`

| CRM (PK) | NOME | EMAIL |
|---|---|---|
| 567890 | MARCELO ARAÚJO | maraujo@fviver.com.br |
| 234891 | ALEXANDRE SOUZA | asouza@fviver.com.br |
| 167384 | JUNIA TRINDADE | jtrindade@fviver.com.br |

`consulta`

| ID (PK) | DATA | HORA | CRMMEDICO (FK) | CPFPACIENTE (FK) | SITUACAO |
|---|---|---|---|---|---|
| 6 | 10/03/2020 | 9:00 | 167384 | 30303030303 | realizada |
| 2 | 10/01/2020 | 10:00 | 234891 | 30303030303 | realizada |
| 5 | 20/02/2020 | 16:00 | 234891 | 20202020202 | realizada |
| 1 | 10/01/2020 | 8:00 | 567890 | 10101010101 | realizada |
| 3 | 15/01/2020 | 8:00 | 567890 | 30303030303 | realizada |
| 4 | 12/02/2020 | 14:00 | 567890 | 10101010101 | realizada |

`atendimento`

| IDCONSULTA (PK/FK) | IDATENDIMENTO (PK) | IDPROCEDIMENTO |
|---|---|---|
| 1 | 1 | 1 |
| 1 | 2 | 2 |
| 2 | 1 | 2 |
| 3 | 1 | 3 |
| 3 | 2 | 2 |
| 4 | 1 | 4 |
| 5 | 1 | 1 |
| 6 | 1 | 2 |

Aplicando o agrupamento:

| NOME | COUNT(*) |
|---|---|
| MARCELO ARAÚJO | 5 |
| ALEXANDRE SOUZA | 2 |
| JUNIA TRINDADE | 1 |

E, empregando a `HAVING` para somente apresentar os médicos que tenham utilizado mais de um procedimento em seus atendimentos, a resposta final seria:

| NOME | COUNT(*) |
|---|---|
| MARCELO ARAÚJO | 5 |
| ALEXANDRE SOUZA | 2 |

- a quantidade de consultas e a quantidade de procedimentos distintos que o médico utilizou em suas consultas, por nome do médico da Clínica Médica Felizes por Viver.

Na construção da consulta, deve-se utilizar a função de agregação `COUNT` para contar tanto as consultas que foram realizadas pelo `medico` quanto o total de `procedimento` utilizados em `atendimento`, mas com a cláusula `DISTINCT` para não repetir a contagem quando se emprega o mesmo procedimento para o mesmo médico. Então, deve-se fazer a junção entre `procedimento`, `consulta` e `atendimento`!

```
SELECT m.nome, COUNT(*) as qtdeConsultas,
    COUNT(distinct a.idProcedimento) as qtdeProc
   FROM PROCEDIMENTO p, CONSULTA c, ATENDIMENTO a
   WHERE a.idconsulta = c.id and p.codigo = a.idProcedimento
    GROUP BY m.nome;
```

As linhas selecionadas para execução da função de agregação, a partir da junção, serão:

`procedimento`

| CODIGO (PK) | DESCRICAO | PRECO |
|---|---|---|
| 1 | CONSULTA | 150,00 |
| 2 | APLICAÇÃO INJEÇÃO | 60,00 |
| 3 | CURATIVO | 40,00 |
| 4 | REVISAO EXAMES | 80,00 |

`medico`

| CRM (PK) | NOME | EMAIL |
|---|---|---|
| 567890 | MARCELO ARAÚJO | maraujo@fviver.com.br |
| 234891 | ALEXANDRE SOUZA | asouza@fviver.com.br |
| 167384 | JUNIA TRINDADE | jtrindade@fviver.com.br |

consulta

| ID (PK) | DATA | HORA | CRMME-DICO (FK) | CPFPACIEN-TE (FK) | SITUACAO |
|---|---|---|---|---|---|
| 6 | 10/03/2020 | 9:00 | 167384 | 30303030303 | realizada |
| 2 | 10/01/2020 | 10:00 | 234891 | 30303030303 | realizada |
| 5 | 20/02/2020 | 16:00 | 234891 | 20202020202 | realizada |
| 1 | 10/01/2020 | 8:00 | 567890 | 10101010101 | realizada |
| 3 | 15/01/2020 | 8:00 | 567890 | 30303030303 | realizada |
| 4 | 12/02/2020 | 14:00 | 567890 | 10101010101 | realizada |

atendimento

| IDCONSULTA (PK/FK) | IDATENDIMENTO (PK) | IDPROCEDIMENTO |
|---|---|---|
| 1 | 1 | 1 |
| 1 | 2 | 2 |
| 2 | 1 | 2 |
| 3 | 1 | 3 |
| 3 | 2 | 2 |
| 4 | 1 | 4 |
| 5 | 1 | 1 |
| 6 | 1 | 2 |

Aplicando o agrupamento com `distinct`, os procedimentos repetidos para o mesmo médico não serão contados:

| NOME | qtdeConsultas | qtdeProc |
|---|---|---|
| MARCELO ARAÚJO | 5 | 4 |
| ALEXANDRE SOUZA | 2 | 2 |
| JUNIA TRINDADE | 1 | 1 |

## Consultas que utilizam subconsultas ou consultas aninhadas

Utilizar subconsultas ou consultas aninhadas associadas às funções de agregação e agrupamento possibilita selecionar a partir da comparação entre uma informação e um conjunto de dados.



### Exemplo

Aqui, desejamos conhecer o nome do paciente mais novo que frequenta a Clínica Médica Felizes por Viver.

Para atender a essa necessidade, deve-se utilizar a função de agregação `MIN(idade)` para comparar a idade de cada paciente com a menor idade de todos os pacientes. Assim, selecionamos a menor idade do conjunto `paciente` e, em seguida, a comparamos com a idade dos pacientes para gerar o resultado.

```
SELECT nome FROM paciente
   WHERE idade in (SELECT MIN(idade) FROM paciente);
```

A subconsulta (`SELECT MIN(idade) FROM paciente`) retornará o valor 67, equivalente à menor idade dos pacientes.

Em seguida, a consulta mais externa será processada, comparando a idade de cada participante com o valor resultado da subconsulta, 67, resultando em:

| NOME |
|---|
| ANTONIO MANOEL |



### Saiba mais

A comparação utilizando a subconsulta pode também ser efetivada com o uso do operador relacional =, quando a execução do `SELECT` na subconsulta resultar em um único valor.

```
SELECT nome FROM paciente
WHERE idade = (SELECT MIN(idade) FROM paciente);
```

## Referências

AGGREGATE Functions. *In*: POSTGRESQL 12.2 Documentation. [*S. l.*]: The PostgreSQL Global Development Group, 2020. 2661 p. Disponível em: https://www.postgresql.org/files/documentation/pdf/12/postgresql-12-A4.pdf. Acesso em: 15 abr. 2020.

AGREGAÇÃO. *In*: DICIO, Dicionário Online de Português. Porto: 7Graus, 2020. Disponível em: https://www.dicio.com.br/agregacao/. Acesso em: 28 abr. 2020

MANNINO, M. V. *Projeto, desenvolvimento de aplicações e administração de banco de dados*. 3. ed. Porto Alegre: AMGH; 2008. 717 p.

SELECT – GROUP BY – Transact-SQL. *Microsoft Docs*, Redmond, 1 Mar. 2019. Disponível em: https://docs.microsoft.com/pt-br/SQL/t-SQL/queries/SELECT-group-by-transact-SQL?view=SQL-server-ver15. Acesso em: 28 abr. 2020

SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. *Sistema de banco de dados*. 3. ed. São Paulo: Makron Books, 1999. 778 p.

## Leituras recomendadas

AMADEU, C. V. (org.). *Banco de dados*. São Paulo: Pearson Education do Brasil, 2014. 208 p.

DATE, C. J. *Introdução a sistemas de banco de dados*. 8. ed. Rio de Janeiro: Campus, 2004. 865 p.

GARCÍA-MOLINA, H.; ULLMAN, J. D.; WIDOM, J. *Database systems*: the complete book. 2. ed. Prentice Hall, 2008. 1203 p.

HEUSER, C. A. *Projeto de banco de dados*. 6. ed. Porto Alegre: Bookman, 2009. (Série Livros Didáticos Informática UFRGS, 4).

MEDEIROS, L. F. *Banco de dados*: princípios e prática. Curitiba: InterSaberes, 2013. 188 p.

RAMAKRISHNAN, R..; GEHRKE, J. *Sistemas de gerenciamento de banco de dados*. 3. ed. Porto Alegre: AMGH; Bookman, 2008. 905 p.

SETZER, V. W.; SILVA, F. S. C. *Banco de dados*: aprenda o que são, melhore seu conhecimento, construa os seus. São Paulo: Blucher, 2005. 392 p.



## Fique atento

Os *links* para *sites* da *web* fornecidos neste capítulo foram todos testados, e seu funcionamento foi comprovado no momento da publicação do material. No entanto, a rede é extremamente dinâmica; suas páginas estão constantemente mudando de local e conteúdo. Assim, os editores declaram não ter qualquer responsabilidade sobre qualidade, precisão ou integralidade das informações referidas em tais *links*.

Encerra aqui o trecho do livro disponibilizado para esta Unidade de Aprendizagem. Na Biblioteca Virtual da Instituição, você encontra a obra na íntegra.

Conteúdo:

SAGAH
SOLUÇÕES EDUCACIONAIS INTEGRADAS

# Dica do Professor: Funções de agregação e agrupamento de dados em SQL

No conjunto de instruções disponibilizado pela SQL, as funções de agregação são funções matemáticas aplicadas ao conjunto de informações. A função de agregação count() é utilizada para apresentar o número de linhas resultado de uma execução de consulta. Sua utilização é importante, pois vai expressar o volume de dados e apurar a quantidade de ocorrências, provendo uma visão quantitativa de informações armazenadas em tabelas.

Nesta Dica do Professor, você verá como utilizar a função de agregação COUNT para obter determinadas informações.

Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

# Saiba mais: Funções de agregação e agrupamento de dados em SQL

Para ampliar o seu conhecimento a respeito desse assunto, veja abaixo as sugestões do professor:

## Modelagem e desenvolvimento de banco de dados

Neste livro, você encontrará outras visões de uso das funções agregadas e de agrupamento, para construir a melhor forma de aplicação.

Conteúdo interativo disponível na plataforma de ensino!

## Oracle Database 11G SQL [Série Oracle]

Nesta obra, você verá como ocorre o uso das instruções SQL no SGBD Oracle.

Conteúdo interativo disponível na plataforma de ensino!

## Microsoft SQL Server 2012 – passo a passo

Nesta publicação, você verá a apresentação do uso das instruções SQL no SGBD SQL SERVER da empresa Microsoft.

Conteúdo interativo disponível na plataforma de ensino!

# Infográfico: Consultas complexas em SQL

Em uma consulta em SQL somente as cláusulas SELECT e FROM são obrigatórias. A cláusula SELECT lista os atributos a serem recuperados, enquanto a cláusula FROM especifica todas as tabelas úteis para a consulta, bem como suas relações. Já a cláusula WHERE é utilizada para especificar as condições para selecionar as tuplas dessas relações, bem como condições de junção, quando necessário. A cláusula GROUP BY especifica os atributos para o agrupamento, enquanto que a cláusula ORDER BY  define uma ordem para a exibição do resultado de uma consulta.

As funções de agregação COUNT, SUM, MIN, MAX e AVG são empregadas em conjunto com o agrupamento. A função COUNT conta o número de tuplas ou valores, conforme for definido na consulta e a COUNT DISTINCT conta somente elementos distintos, enquanto a função SUM possibilita a soma de valores; MAX é empregada para verificar um valor máximo; MIN para um valor mínimo e a AVG para uma média de valores.

Neste Infográfico, você vai ver uma representação gráfica de algumas destas cláusulas e funções para facilitar sua compreensão.

**A imagem a seguir possui audiodescrição. Para acessar o recurso,** clique aqui

# CLÁUSULAS E FUNÇÕES DE AGREGAÇÃO SQL

O objetivo deste Infográfico é tornar o conteúdo abordado na disciplina mais visível e compreensível com imagens e exemplos simples, além dos textos e exemplos empregados previamente.

Veja a descrição e os exemplos práticos das principais cláusulas e funções de agregação utilizadas para a criação de consultas em SQL.



## SELECT

Seleciona dados em uma banco de dados.

Exemplo:

```
SELECT nome FROM alunos;
```

## FROM

Especifica todas as tabelas e/ou relações úteis para a consulta.

Exemplo:

```
SELECT nome FROM alunos;
```





## WHERE

Especifica condições na consulta com base em operações específicas.

Exemplo:

```
SELECT nome FROM alunos
WHERE idade=20;
```

## ORDER BY

Ordena o resultado da consulta pela coluna especificada.

Exemplo:

```
SELECT * FROM alunos
ORDER BY nome;
```





## GROUP BY

Agrupa o resultado de funções agregadas no SELECT pela coluna que for especificada.

Exemplo:

```
SELECT faculdade, COUNT (*) FROM alunos
GROUP BY faculdade;
```

## AND E OR

O operador AND determina que mais de uma condição especificada na operação seja verdadeira e o OR que ao menos uma das condições especificadas na operação seja verdadeira.

Exemplo:

```
SELECT CPF FROM alunos
WHERE matricula=1234 AND idade=22

SELECT CPF FROM alunos
WHERE matricula=1234 OR idade=22
```





## COUNT

Conta o número de linhas em uma consulta.

Exemplo:

```
SELECT COUNT (*) FROM faculdade;
```

## *SUM*

Soma os valores de uma seleção.

Exemplo:

```
SELECT SUM (valor) FROM produto;
```





## AVG

Retorna a média dos valores de uma seleção.

Exemplo:

```
SELECT AVG (valor) FROM produto;
```

## MAX E MIN

Retornam o maior (MAX) e o menor (MIN) valor de uma seleção.

Exemplo:

```
SELECT MAX (valor) FROM produto;

SELECT MAX (valor) FROM produto;
```



Essas são apenas algumas das possibilidades de utilização de cláusulas e funções, existem ainda muitas outras possíveis.

Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

# Conteúdo do Livro: Consultas complexas em SQL

Bancos de dados são muito úteis para empresas, pois garantem organização, controle e segurança de suas informações. Por isso, é evidente sua importância para o bom funcionamento das atividades dessas organizações e para que suas missões e visões sejam alcançadas. A fim de que isso seja possível, é necessário que você, profissional de Tecnologia da Informação, tenha conhecimento sobre este tipo de ferramenta, trabalhando nestas questões como analista contratado ou como proprietário de sua própria empresa.

No capítulo Consultas complexas em SQL, do livro *Banco de Dados*, base teórica desta Unidade de Aprendizagem, você vai estudar sobre consultas complexas em SQL. Além disso, vai ver como os blocos com SELECT, FROM e WHERE completos dentro de uma cláusula WHERE podem ser utilizados ao longo de outra consulta, a chamado de consulta aninhada.

Boa leitura.

**Os elementos gráficos deste capítulo possuem audiodescrição. Para acessar o recurso,** clique aqui

# BANCO DE DADOS

**Roni Francisco Pichetti**



SAGAH
SOLUÇÕES EDUCACIONAIS INTEGRADAS

# Consultas complexas em SQL

## Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Descrever funções de agregação e agrupamento na linguagem SQL.
- Desenvolver consultas avançadas em banco de dados usando junção, agregação e ordenação.
- Produzir consultas avançadas empregando conceitos de aninhamento e correlacionamento.

## Introdução

Os bancos de dados podem armazenar informações sobre diferentes áreas do conhecimento, como no controle de estoques ou no cadastro de colaboradores em uma organização. Por exemplo, considerando os dados dos colaboradores, esses bancos abrangem suas informações pessoais, seu relacionamento com a empresa (setor, departamento, supervisão), bem como salários, benefícios e auxílios. Muitos tipos e formas de consultas podem ser realizadas em um banco de dados que utiliza a linguagem de consulta estruturada (SQL, do inglês *structured query language*) com essas informações. Algumas funções podem agilizar o trabalho do usuário final, como realizar somas de valores, calcular médias ou contar registros de colaboradores pela própria linguagem de consulta.

Neste capítulo, você vai estudar as funções de agregação e agrupamento em SQL. Você também vai conferir exemplos da criação de consultas em SQL utilizando junção, agregação e ordenação e, ainda, vai aprender sobre consultas avançadas empregando aninhamento e correlacionamento.

# 1 Funções de agregação e agrupamento em SQL

As funções em SQL são recursos que facilitam diferentes tarefas consideradas corriqueiras. Elas são divididas em grupos de características operacionais, como manipulação de datas, operações matemáticas, entre outras. As **funções de agregação** são utilizadas para resumir informações de diferentes tuplas em uma síntese de tuplas única. Conforme lecionam Ramarkrishnan e Gehrke (2011), além de obter dados simples, muitas vezes, é necessário aplicar pesquisas com resumos ou computações em grupos de dados. Assim, o agrupamento é empregado para criar subgrupos de tuplas antes do resumo.

O agrupamento e a agregação são necessários em diferentes aplicações de bancos de dados. Entre as funções de agregação embutidas, estão: `COUNT`, `COUNT DISTINCT`, `SUM`, `MAX`, `MIN` e `AVG`. De acordo com Manzano (2017), a sintaxe da definição e do uso das funções de agregação é a seguinte:

```
NOME _ FUNÇÃO ([ALL / DISTINCT] / [expressão]);
```

onde o `NOME_FUNÇÃO` se refere à definição de uma função de agregação, e `ALL` é responsável por aplicar a função de agregação em todos os valores da expressão. Já `DISTINCT` aplica a ação da função uma única vez em cada instância, e `expressão` trata do tipo de valor obtido no campo de um registro, que pode ser numérico, por exemplo (MANZANO, 2017).

A função `COUNT` conta o número de tuplas ou valores, conforme for definido na consulta, e a função `COUNT DISTINCT` conta somente elementos distintos. Já a função `SUM` possibilita a soma de valores, `MAX` é empregada para verificar um valor máximo, `MIN` é utilizada para verificar um valor mínimo, e `AVG` é usada para verificar uma média de valores. Essas funções podem ser aplicadas tanto em um conjunto quanto em um multiconjunto de valores. As funções `MAX` e `MIN` também podem ser utilizadas com atributos que possuem domínios não numéricos, desde que esses valores estejam em uma ordenação total entre si, como os domínios `DATE` e `TIME` (ELMASRI; NAVATHE, 2011).

## Exemplo

A seguir, você pode conferir um exemplo de consulta com funções de agregação.

```
SELECT SUM (Salario), MAX (Salario), MIN (Salario), AVG
(Salario)
FROM COLABORADOR;
```

Com a consulta do exemplo, verifica-se a soma, os valores máximo e mínimo e a média dos salários dos colaboradores cadastrados no banco de dados. Caso seja necessário obter os valores separados de um departamento específico, pode ser utilizada a cláusula `WHERE`. Assim, os resultados da consulta sobre os salários dos colaboradores serão restringidos pelo departamento que for descrito na cláusula `WHERE` (ELMASRI; NAVATHE, 2011).

O exemplo a seguir traz uma consulta em que é utilizada a função `COUNT`:

```
SELECT COUNT (*)
FROM COLABORADOR, DEPARTAMENTO
WHERE Dnr=dnumero
    AND Dnome='Faturamento';
```

Nesse caso, a consulta trata de verificar e contar o número de colaboradores cadastrados no banco de dados que fazem parte do departamento `'Faturamento'`. Veja que o asterisco (*) se refere a todas as tuplas da tabela, de modo que `COUNT (*)` resulta no número total de linhas com a informação solicitada, e não de seus atributos. Caso seja necessário utilizar a função `COUNT` em valores de uma única coluna, ela pode ser especificada na função. Se empregada aliada ao `DISTINCT`, os valores duplicados serão eliminados.

Cabe ressaltar que quaisquer tuplas com valor nulo (`NULL`), ao se utilizar essa função, não serão contadas, visto que, geralmente, os valores `NULL` são descartados quando as funções de agregação são aplicadas a uma coluna. Portanto, pode-se compreender que o uso das funções de agregação gera um resumo da parte do banco de dados que for consultada, conforme lecionam Elmasri e Navathe (2011).

Existe a possibilidade de se aplicar funções de agregação a subgrupos de tuplas em uma relação, como o salário médio dos colaboradores de cada departamento da empresa ou o número de colaboradores envolvidos no trabalho de cada projeto. Para isso, é preciso dividir a relação em subconjuntos de tuplas não sobrepostos. Cada partição será formada pelas tuplas que possuírem o mesmo valor em um ou mais atributos, os quais são chamados de **atributos de agrupamento**. Assim, a função pode ser aplicada a cada grupo de forma independente, para se produzir um resumo. A SQL possui a cláusula `GROUP BY`, útil nesses casos (ELMASRI; NAVATHE, 2011).

De acordo com Ramarkrishnan e Gehrke (2011), a cláusula `GROUP BY` é empregada nos casos em que é preciso aplicar operações agregadas a cada um dos vários grupos de tuplas em uma relação. Dessa forma, essa cláusula especifica os atributos de agrupamento, para que o valor resultante do emprego de cada função de agregação a um grupo apareça junto com o valor dos seus atributos.



### Exemplo

Veja um exemplo do uso da cláusula `GROUP BY` (ELMASRI; NAVATHE, 2011):

```
SELECT Dnr, COUNT (*), AVG (Salario)
FROM COLABORADOR
GROUP BY Dnr;
```

A consulta do exemplo procura recuperar o número do departamento, o número de colaboradores no departamento e o salário médio em cada departamento. Para isso, as tuplas `COLABORADOR` são divididas em grupos pelo valor do atributo `Dnr`, que é o número do departamento. Assim, cada grupo contém os colaboradores que trabalham no departamento determinado. As funções `COUNT` e `AVG` são aplicadas separadamente em cada grupo de tuplas. Caso houver um ou mais valores `NULL` no atributo que será utilizado para o agrupamento, é criado um grupo separado para todas essas tuplas (ELMASRI; NAVATHE, 2011). Para facilitar a compreensão, nos quadros a seguir é apresentado o resultado da consulta do exemplo com `GROUP BY` em forma de tabela.

| Pnome | ... | Unome | Cpf | Salario | Dnr |
|---|---|---|---|---|---|
| Diego | | Silva | 12345678966 | 3.000 | 5 |
| Paulo | | Souza | 33344455566 | 2.000 | 5 |
| Ana | | Freitas | 66677788899 | 10.000 | 5 |
| Lia | | Martins | 98765432199 | 7.500 | 4 |
| Jorge | | Lima | 98798798798 | 4.500 | 4 |
| João | | Brito | 87687687676 | 6.000 | 4 |
| Regina | | Pereira | 54545454545 | 15.000 | 1 |

*Fonte:* Adaptado de Elmasri e Navathe (2011).

| Dnr | COUNT(*) | AVG (Salario) |
|---|---|---|
| 5 | 3 | 5.000 |
| 4 | 3 | 6.000 |
| 1 | 1 | 15.000 |

*Fonte:* Adaptado de Elmasri e Navathe (2011).

De acordo com os dados presentes nos quadros, é possível verificar que a consulta criou três grupos distintos, de acordo com o número do departamento, e aplicou as funções `COUNT` e `AVG` às respectivas tuplas componentes dos grupos. A coluna sem informações mostra que a tabela `COLABORADORES` possui mais dados, que não estão visíveis no exemplo.

O próximo exemplo trata de uma consulta sobre o número e o nome de projetos e o número de colaboradores que trabalham em cada projeto, utilizando agrupamento.



**Exemplo**

Veja o código a seguir (ELMASRI; NAVATHE, 2011):

```
SELECT Projnumero, Projnome, COUNT (*)
FROM PROJETO, TRABALHA _ EM
WHERE Projnumero=Pnr
GROUP BY Projnumero, Projnome;
```

Neste exemplo, fica evidente a possibilidade de utilizar uma condição de junção em um conjunto `GROUP BY`, visto que o agrupamento e as funções são aplicadas depois da junção das duas relações. Pode ser necessário aplicar funções somente a grupos que satisfazem determinadas condições. Levando-se em conta o exemplo dos números de projetos, pode ser necessário pesquisar apenas os projetos que contam com mais de dois colaboradores. Para isso, a SQL possui a cláusula `HAVING`, que pode ser empregada junto à cláusula `GROUP BY`. A cláusula `HAVING` possibilita incluir uma condição sobre a informação de resumo referente ao grupo de tuplas criado pela cláusula `GROUP BY`. Dessa forma, somente os grupos que contemplarem a condição serão recuperados no resultado da consulta (ELMASRI; NAVATHE, 2011).

## 2 Consultas avançadas em banco de dados usando junção, agregação e ordenação

Uma consulta em SQL pode possuir até seis cláusulas, mas somente as cláusulas `SELECT` e `FROM` são obrigatórias. A consulta pode ser dividida em várias linhas e deve terminar com um sinal de ponto e vírgula. Os termos de uma consulta são separados por espaços, sendo que parênteses podem ser utilizados para agrupar partes relevantes de uma consulta.

A cláusula `SELECT` lista os atributos a serem recuperados, enquanto a cláusula `FROM` especifica todas as tabelas úteis para a consulta, bem como suas relações. Já a cláusula `WHERE` é utilizada para especificar as condições para selecionar as tuplas dessas relações, bem como condições de junção, quando necessário. A cláusula `GROUP BY` especifica os atributos para o agrupamento, e a cláusula `HAVING`, uma condição sobre os grupos selecionados. As funções de agregação `COUNT`, `SUM`, `MIN`, `MAX` e `AVG` são empregadas em conjunto

com o agrupamento e também podem ser aplicadas em uma consulta sem agrupamento. A cláusula `ORDER BY`, por sua vez, define uma ordem para a exibição do resultado de uma consulta (ELMASRI; NAVATHE, 2011).

> Para formular consultas de maneira correta, é útil considerar as etapas que definem o significado ou a semântica de cada consulta. Uma consulta é avaliada conceitualmente aplicando primeiro a cláusula `FROM` (para identificar todas as tabelas envolvidas na consulta ou materializar quaisquer tabelas de junção), seguida pela cláusula `WHERE` para selecionar e juntar tuplas, e depois por `GROUP BY` e `HAVING`. Conceitualmente, `ORDER BY` é aplicado no final para classificar o resultado da consulta. Se nenhuma das três últimas cláusulas (`GROUP BY`, `HAVING` e `ORDER BY`) for especificada, podemos pensar conceitualmente em uma consulta como sendo executada da seguinte forma: para cada combinação de tuplas, [...] avaliar a cláusula `WHERE`; se ela for avaliada como `TRUE`, colocar os valores dos atributos especificados na cláusula `SELECT` dessa combinação de tuplas no resultado da consulta. (ELMASRI; NAVATHE, 2011, p. 86).

Existem várias formas de especificar a mesma consulta em SQL. Uma vantagem dessa flexibilidade é que os usuários podem escolher a técnica com a qual estão mais acostumados para definir uma consulta, enquanto, para o programador do sistema, a fim de otimizar a consulta, é preferível que uma consulta tenha o mínimo de ordenação possível, por exemplo. Uma desvantagem de se ter diferentes maneiras de escrever a mesma consulta é que isso pode criar confusão para o usuário, que pode não saber qual é a técnica correta de consulta para cada situação. Além disso, ainda que seja mais eficiente executar uma consulta de uma forma específica, essa forma pode não ser empregada. De qualquer forma, o ideal é que o usuário se preocupe em especificar a consulta corretamente, enquanto o sistema de gerenciamento de banco de dados determina como colocar em prática a consulta de maneira eficiente, conforme lecionam Elmasri e Navathe (2011).

É necessário ter cuidado quando duas condições diferentes se aplicam em uma consulta, para que uma não se sobreponha à outra, como quando utilizadas as cláusulas `WHERE` e `HAVING` em sequência (ELMASRI; NAVATHE, 2011). De acordo com Ramarkrishnan e Gehrke (2011), `HAVING` é uma cláusula opcional, que pode ser empregada para especificar qualificações nos grupos definidos pelo uso da cláusula `GROUP BY`.



## Exemplo

Verifique a seguir um exemplo incorreto e um exemplo correto das consultas com agregação propostas (ELMASRI; NAVATHE, 2011).

**Consulta incorreta:**

```
SELECT Dnome, COUNT (*)
FROM DEPARTAMENTO, COLABORADOR
WHERE Dnumero=Dnr AND Salario>5.000
GROUP BY Dnome;
HAVING COUNT (*) > 5;
```

**Consulta correta:**

```
SELECT Dnumero, COUNT (*)
FROM DEPARTAMENTO, COLABORADOR
WHERE Dnumero=Dnr AND Salario>5.000 AND
     (SELECT Dnr IN
     FROM COLABORADOR
     GROUP BY DnR
     HAVING COUNT (*) > 5);
```

A primeira consulta do exemplo está incorreta, pois vai selecionar somente os departamentos que tenham mais de cinco funcionários, cada um com um salário de mais de R$ 5.000. Sabe-se que a cláusula `WHERE` é executada primeiro, a fim de selecionar as tuplas individuais ou tuplas de junção; assim, a cláusula `HAVING` é aplicada depois, para selecionar os grupos individuais de tuplas. Dessa forma, as tuplas já estarão restritas a colaboradores que recebem mais de R$ 5.000 antes que a cláusula `HAVING` possa ser aplicada, e o resultado obtido não será o esperado. Dito isso, uma das formas corretas de realizar essa mesma consulta é com o uso de consulta aninhada, que será detalhada na sequência. O que foi feito para que a cláusula fosse aplicada integralmente foi alterar a ordem da pesquisa. Dessa forma, é possível consultar primeiro o departamento que possui mais de cinco funcionários e, depois, o número do departamento e de colaboradores com salário maior do que R$ 5.000 (ELMASRI; NAVATHE, 2011).

Consultas avançadas podem ser realizadas em SQL por meio de junções. A SQL possui suporte para operações de junção de diferentes tipos, entre elas:

- a `INNER JOIN`, que trata de uma junção interna de duas ou mais tabelas, na qual uma tupla somente é incluída no resultado se combinar com a outra relação; e
- a `NATURAL JOIN`, por meio da qual cada par de atributos é incluído apenas uma vez na relação resultante.

Caso os nomes dos atributos de junção não sejam os mesmos nas relações iniciais, é possível alterar seus nomes para que combinem e aplicar a `NATURAL JOIN` somente depois. Se o resultado for nulo para uma das relações, a tupla é excluída da tabela de junção.

Existem diferentes opções disponíveis para distinguir tabelas de junção; entre elas, estão:

- `LEFT OUTER JOIN`, na qual toda tupla da tabela da esquerda precisa estar presente no resultado, caso contrário, é preenchida com valores nulos;
- `RIGHT OUTER JOIN`, em que toda tupla da tabela da direita precisa estar presente no resultado, caso contrário, é preenchida com valores nulos.

Os próximos exemplos apresentam consultas utilizando junções internas e ordenação.



**Exemplo**

Veja, a seguir, uma consulta sobre o nome e o endereço de todos os colaboradores que trabalham no departamento de `Compras` utilizando `JOIN` (ELMASRI; NAVATHE, 2011).

```
SELECT Pnome, Unome, Endereco
FROM (COLABORADOR JOIN DEPARTAMENTO ON Dnr=Dnumero)
WHERE Dnome= 'Compras'
ORDER BY Pnome, Unome;
```

A seguir, é apresentada uma consulta em que a relação DEPARTAMENTO é renomeada como DEP e seus atributos são renomeados para combinar com os da tabela Colaborador, utilizando NATURAL JOIN.

```
SELECT Pnome, Unome, Endereco
FROM (COLABORADOR NATURAL JOIN
     (DEPARTAMENTO AS DEP
     (Dnome, Dnr, Cpf _ gerente)))
WHERE Dnome= 'Compras'
ORDER BY Pnome, Unome;
```

Nesses exemplos, pode ser observado que os resultados das consultas com junções serão apresentados de forma ordenada, pelo primeiro nome e pelo último nome, respectivamente, por conta do uso da cláusula ORDER BY.

As operações de junção também podem ser utilizadas combinadas às funções de agrupamento e agregação, conforme pode ser observado no próximo exemplo.



**Exemplo**

Veja, a seguir, uma consulta sobre a soma dos salários de todos os colaboradores vinculados ao departamento de Compras, assim como os salários máximo e mínimo e a média dos salários desse departamento (ELMASRI; NAVATHE, 2011).

```
SELECT SUM (Salario), MAX (Salario), MIN (Salario), AVG
(Salario)
FROM (COLABORADOR JOIN DEPARTAMENTO ON Dnr=Dnumero)
WHERE Dnome= 'Compras';
```

## 3 Consultas avançadas empregando conceitos de aninhamento e correlacionamento

Determinadas consultas necessitam que os valores presentes no banco de dados sejam buscados e também comparados. Esse tipo de consulta pode ser formulado utilizando-se **consultas aninhadas**, que são blocos com `SELECT`, `FROM` e `WHERE` completos dentro de uma cláusula `WHERE` de outra consulta. A outra consulta é chamada, nesse caso, de **consulta externa** (ELMASRI; NAVATHE, 2011). De acordo com Ramarkrishnan e Gehrke (2011), as consultas aninhadas são um dos recursos mais poderosos da SQL. Os autores consideram a consulta aninhada aquela que possui outra ou outras consultas embutidas dentro dela, sendo que a consulta embutida é chamada de **subconsulta**.



### Exemplo

Veja, a seguir, um exemplo de consulta aninhada (ELMASRI; NAVATHE, 2011).

```
SELECT DISTINCT Projnumero
FROM PROJETO
WHERE Projnumero IN
      (SELECT Projnumero
      FROM PROJETO,
            DEPARTAMENTO,
            FUNCIONARIO
            Dnum=Dnumero AND
      WHERE Cpf _ gerente=Cpf
            AND Unome='Silva')
      OR Projnumero IN
      (SELECT Pnr
      FROM TRABALHA _ EM,
            FUNCIONARIO
      WHERE Fcpf=Cpf AND
            Unome='Silva');
```

A primeira consulta aninhada seleciona números de projetos que possuem um funcionário cujo último nome (sobrenome) é igual a Silva e que seja gerente. Já na segunda consulta aninhada, são selecionados os números dos projetos com um funcionário também de sobrenome Silva, mas que tenha o vínculo de trabalhador. Na consulta externa, é empregado o conectivo lógico `OR`, a fim de que o resultado possa apresentar um `PROJETO` cujo `PROJNUMERO` esteja em qualquer uma das consultas aninhadas. De forma geral, a consulta aninhada resulta em uma tabela que é um conjunto ou multiconjunto de tuplas (ELMASRI; NAVATHE, 2011).

Quando uma consulta aninhada retorna um único atributo e uma tupla única, o resultado é um único valor. Nesse caso, pode-se utilizar o sinal = em vez do `IN` para o operador de comparação. De qualquer forma, geralmente a consulta aninhada resulta em uma tabela, que é formada por um conjunto de tuplas. As consultas podem estar aninhadas em vários níveis. Por conta disso, é necessário lidar com a possibilidade de ambiguidade entre nomes de atributos que possuam a mesma descrição. A regra, nesses casos, é que uma referência a um atributo não qualificado se refere à relação declarada na consulta mais interna. Essa regra é semelhante às regras de escopo para variáveis de programa em diferentes linguagens de programação, conforme apontam Elmasri e Navathe (2011).



**Exemplo**

O próximo exemplo trata da ambiguidade em potencial de nomes de atributo nas consultas aninhadas (ELMASRI; NAVATHE, 2011).

```
SELECT C.Pnome, C.Unome
FROM COLABORADOR AS C
WHERE C.Cpf IN
      (SELECT D.Ccpf
      FROM DEPENDENTE AS D
      WHERE C.Pnome=D.Niome _ dependente
            AND C.Sexo=D.Sexo);
```

A consulta do exemplo busca recuperar o nome de cada colaborador que possui um dependente com o mesmo nome e com o mesmo sexo do funcionário. Na consulta aninhada do exemplo, foi necessário qualificar `C.Sexo`, pois se refere ao atributo `Sexo` de `COLABORADOR` na consulta externa, visto que `DEPENDENTE` também possui o mesmo atributo, com a mesma nomenclatura. Caso houvesse alguma referência não qualificada ao atributo `Sexo` na consulta aninhada, eles se refeririam ao atributo de `DEPENDENTE`. Já a qualificação dos atributos `Pnome` e `Cpf` de `COLABORADOR` não seria necessária, já que a relação `DEPENDENTE` não possui atributos com os mesmos nomes. Para evitar erros e ambiguidade, é recomendável criar variáveis de tupla para todas as referências em uma consulta, conforme realizado na consulta do exemplo (ELMASRI; NAVATHE, 2011).

As consultas aninhadas podem ser consideradas **correlacionadas**, quando uma condição da cláusula `WHERE` de uma consulta aninhada faz referência a algum atributo de uma relação declarada na consulta externa. Para compreender essa correlação, pense que a consulta aninhada é avaliada uma vez para cada linha (ou tupla) na consulta externa. Levando-se em conta o exemplo anterior, para cada tupla de `COLABORADOR`, a consulta aninhada recupera os valores de `Ccpf` para todas as tuplas de `DEPENDENTE` com o mesmo sexo e nome da tupla da tabela de colaboradores. Caso o valor do `Ccpf` estiver no resultado da consulta aninhada, então, é selecionada essa tupla em `COLABORADOR`. De forma geral, uma consulta escrita com blocos aninhados, utilizando as cláusulas `SELECT`, `FROM` e `WHERE` e usando operadores de comparação = ou `IN`, sempre pode ser definida com uma única consulta em bloco (ELMASRI; NAVATHE, 2011).

A linguagem SQL possui um grande número de recursos para que seus usuários possam especificar recuperações complexas do banco de dados. Nesse sentido, vale ressaltar a importância das regras para se lidar com valores `NULL` (nulos). A expressão `NULL` é utilizada para identificar valores que estão faltando, os quais podem ser desconhecidos, não disponíveis ou, ainda, não aplicáveis. Geralmente, cada `NULL` é considerado diferente de qualquer outro presente nos registros do banco de dados. Assim, quando um valor `NULL` passa por uma operação de comparação, o resultado acaba sendo considerado `UNKNOWN` (desconhecido), `TRUE` (verdadeiro) ou `FALSE` (falso). Portanto, a linguagem SQL utiliza uma **lógica de três valores**, com `TRUE`, `FALSE` e `UNKNOWN`, em vez da lógica booleana padrão, que emprega apenas dois valores. Dessa forma, é necessário definir os três valores verdadeiros das expressões

que utilizarem os conectivos lógicos AND, OR e NOT. O Quadro 1 apresenta os valores resultantes possíveis nos três casos (ELMASRI; NAVATHE, 2011).

**Quadro 1.** Conectivos lógicos na lógica de três valores

| AND | TRUE | FALSE | UNKNOWN |
|---|---|---|---|
| TRUE | TRUE | FALSE | UNKNOWN |
| FALSE | FALSE | FALSE | FALSE |
| UNKNOWN | UNKNOWN | FALSE | UNKNOWN |

| OR | TRUE | FALSE | UNKNOWN |
|---|---|---|---|
| TRUE | TRUE | TRUE | TRUE |
| FALSE | TRUE | FALSE | UNKNOWN |
| UNKNOWN | TRUE | UNKNOWN | UNKNOWN |

| NOT | — | — | — |
|---|---|---|---|
| TRUE | FALSE | — | — |
| FALSE | TRUE | — | — |
| UNKNOWN | UNKNOWN | — | — |

*Fonte:* Adaptado de Elmasri e Navathe (2011).

No Quadro 1, as linhas e colunas representam os valores dos resultados em condições de comparação que geralmente são utilizados na cláusula WHERE de uma consulta SQL. A combinação de dois valores utilizando o conectivo AND é mostrada nas primeiras quatro linhas. Em sequência, é apresentado o resultado do uso do conectivo OR. Por exemplo, o resultado de (TRUE AND FALSE) é FALSE, enquanto o resultado de (TRUE OR FALSE) é TRUE. Já no caso dos resultados da operação lógica NOT, apresentados ao final do primeiro grupo do Quadro 1, apenas há mudança no TRUE e no FALSE, enquanto o UNKNOWN permanece igual.

A SQL possibilitar realizar consultas que verificam se o valor de um atributo é `NULL` utilizando os operadores de comparação `IS` ou `IS NOT`. Isso se aplica porque se compreende cada valor nulo como diferente de outro valor nulo. Assim, a comparação da igualdade não se torna apropriada (ELMASRI; NAVATHE, 2011).



### *Link*

Você viu, ao longo deste capítulo, diferentes conceitos e exemplos em SQL de consultas utilizando junção, agregação e ordenação. Acessando o *link* a seguir, você pode conferir conceitos adicionais sobre funções de agregação, que podem ser empregadas em diferentes versões de bancos de dados em SQL.

**https://qrgo.page.link/P115X**



### Referências

ELMASRI, R.; NAVATHE, S. B. *Sistemas de banco de dados*. 6. ed. São Paulo: Pearson, 2011.

MANZANO, J. A. N. G. *Microsoft SQL Server 2016*: express edition interativo. São Paulo: Érica, 2017.

RAMARKRISHNAN, R.; GEHRKE, J. *Sistemas de gerenciamento de banco de dados*. 3. ed. Porto Alegre: AMGH, 2011.



### Fique atento

Os *links* para *sites* da *web* fornecidos neste capítulo foram todos testados, e seu funcionamento foi comprovado no momento da publicação do material. No entanto, a rede é extremamente dinâmica; suas páginas estão constantemente mudando de local e conteúdo. Assim, os editores declaram não ter qualquer responsabilidade sobre qualidade, precisão ou integralidade das informações referidas em tais *links*.

Encerra aqui o trecho do livro disponibilizado para esta Unidade de Aprendizagem. Na Biblioteca Virtual da Instituição, você encontra a obra na íntegra.

Conteúdo:

SAGAH
SOLUÇÕES EDUCACIONAIS INTEGRADAS

# Dica do Professor: Consultas complexas em SQL

As consultas em um banco de dados auxiliam na realização das tarefas tanto dos setores de Tecnologia de Informação, muitas vezes responsáveis pela garantia de sua integridade, quanto dos demais setores de uma empresa.

Nesta Dica do Professor, você vai verificar alguns exemplos de consultas avançadas em SQL.

**As imagens do vídeo a seguir possuem audiodescrição. Para acessar o recurso,** clique aqui

Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

# Saiba mais: Consultas complexas em SQL

Para ampliar o seu conhecimento a respeito desse assunto, veja abaixo as sugestões do professor:

## Funções agregadas

Para aprofundar seus conhecimentos sobre funções agregadas em SQL, assista ao vídeo a seguir, o qual aborda consultas na teoria e na prática em um banco de dados.

Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

## Consultas aninhadas em SQL

Para verificar mais exemplos sobre as consultas aninhadas em SQL, consulte o seguinte livro. Dê atenção especial aos conceitos presentes entre as páginas 122 a 127, as quais tratam especificamente sobre consultas aninhadas.

Conteúdo interativo disponível na plataforma de ensino!

## Consultas avançadas com SQL

Para verificar mais exemplos sobre as consultas avançadas em SQL, consulte o seguinte livro. Dê atenção especial aos conceitos presentes entre as páginas 298 a 303, que tratam especificamente sobre junções internas e externas.

Conteúdo interativo disponível na plataforma de ensino!

## Conceitos de agregação e agrupamento

Confira no *link* a seguir a documentação oficial do MySQL sobre funções de agregação e agrupamento, que traz exemplos da sua aplicação em banco de dados.



Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

# Infográfico: Índices em banco de dados

O SGBD que você está estudando é o PostgreSQL. Ele fornece vários tipos de índices disponíveis para a indexação de colunas em uma tabela de um banco de dados. Cada um desses tipos deve ser utilizado dependendo das características do banco de dados e do tipo de consulta desejada. Quando você projetar um banco de dados, deve se perguntar que tipo de consulta vai ser necessário realizar para, depois, escolher o tipo de índice e a coluna, ou colunas, a indexar.

Acompanhe o Infográfico para ver que tipos de dados é possível indexar para cada tipo de índice do PostgreSQL. Confira também algumas vantagens do uso desse SGBD.

# ÍNDICES NO POSTGRESQL

Existem vários tipos de índices fornecidos pelo SGBD PostgreSQL, sendo que cada um usa um algoritmo diferente. O tipo de índice que você criar vai depender do tipo de consultas que você deseja realizar no banco de dados.



**Para criar um índice no PostgreSQL, utilize o comando:**

```
CREATE INDEX nome ON tabela USING TIPO (coluna);
```

***nome:*** escolha um nome para o índice.
***tabela:*** nome da tabela em que você deseja criar um índice.
***TIPO:*** tipo de índice que você deseja criar.
***coluna:*** nome da coluna em que o índice vai ser criado.

## TIPOS DE ÍNDICE

Veja, a seguir, os tipos de índice disponíveis no PostgreSQL:

### ÁRVORE B

- Padrão do PostgreSQL quando o TIPO não é definido.
- Pode ser usada em consultas usando qualidade, intervalos e predicados nulos.
- Pode indexar todos os tipos de dados do PostgreSQL.

### HASH

- Suportado a partir da versão 10 do PostgreSQL.
- É útil para consultas com comparações de igualdade.

### GIN

- É útil para acelerar consultas em estruturas de dados complexas, como matrizes e pesquisas de texto completo.
- É útil quando vários valores precisam mapear para uma linha da tabela.
- A consulta é 3x mais rápida em relação ao uso de índice GIST.

### GIST

- É útil na indexação de tipos de dados geométricos, como pesquisas de texto completo.
- É 3x menor que o índice GIN.
- É 10x mais rápido de atualizar que o índice GIN.

### SP-GIST

- É similar ao GiST.
- É projetado para lidar com tipos de dados complexos definidos pelo usuário.
- Requer mais conhecimento técnico para utilizar.

### BRIN

- É suportado a partir da versão 9.5.
- É útil para tabelas muito grandes, em que o tamanho é limitado.
- A consulta é mais lenta se comparada com Árvore B, mas requer menos espaço em disco que ela.

## VANTAGENS

Veja as vantagens do PostgreSQL em relação a outros SGBD:

### GRATUITO

- Você pode fazer o *download* do PostgreSQL de forma gratuita na Internet.

### DOCUMENTAÇÃO

- O PostgreSQL tem vasta e detalhada documentação disponível para o usuário na Internet.

### *OPEN SOURCE*

- O código do PostgreSQL é aberto para você contribuir no desenvolvimento.

Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

# Conteúdo do Livro: Índices em banco de dados

A procura por uma informação em uma revista ou livro seria muito demorada se você verificasse todas as páginas até encontrá-la. A forma mais eficiente para encontrar o que deseja é utilizar o índice disponível, o qual informa em qual página a informação está. Da mesma forma, a busca em um banco de dados pode se tornar extremamente custosa conforme aumenta o tamanho da tabela com os registros de dados. Assim como nas revistas e nos livros, índices podem ser utilizados para organizar os registros e acelerar a busca pela informação desejada em um banco de dados.

Para entender mais sobre esse assunto, leia o capítulo Índices em banco de dados, da obra *Consultas em banco de dados*, base teórica desta Unidade de Aprendizagem.

Boa leitura.

# CONSULTAS EM BANCO DE DADOS

Gabriella Lopes Andrade



SAGAH
SOLUÇÕES EDUCACIONAIS INTEGRADAS

# Índices em banco de dados

## Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Definir o conceito de índices em banco de dados e suas vantagens.
- Reconhecer como criar e excluir índices em banco de dados.
- Aplicar índices em SQL.

## Introdução

Um dos principais objetivos da criação de um banco de dados consiste em armazenar informações e recuperá-las por meio de consultas ao banco de dados criado. À medida que o tamanho das tabelas aumenta, as consultas em bancos de dados começam a ficar mais dispendiosas e as informações que você deseja recuperar em determinada tabela, por sua vez, demorarão mais tempo para ser encontradas. Uma opção para solucionar esse problema refere-se à utilização de dados chamados de índices, muito úteis para organizar os registros de dados com o objetivo de otimizar as consultas e torná-las mais eficientes. Os índices contêm ponteiros para a tabela com os dados reais do banco de dados, tornando a busca e a recuperação da informação mais rápidas e eficientes.

Neste capítulo, você aprenderá sobre índices em banco de dados, quais as vantagens ao utilizá-los e como eles funcionam teoricamente. Ainda, identificará quais são os tipos de índices disponíveis no Sistema de Gerenciamento de Banco de Dados (SGBD) empregado no capítulo, como criá-los, excluí-los e utilizá-los a partir de exemplos de código.

# 1 Conceito e vantagens

Suponha que você esteja estudando sobre determinado tema e deseje encontrar informações sobre ele em um livro: você pode percorrer todo o livro até encontrar a informação desejada ou, então, utilizar o índice do livro para encontrar a página na qual ela está localizada. Percorrer o livro inteiro com certeza demandará muito tempo, uma busca que é facilitada pelo índice. Um índice de um livro é semelhante a um índice em um banco de dados, exceto pelo fato de que utilizamos o segundo para encontrar linhas específicas em uma tabela (PRICE, 2009).

Ao empregarmos uma definição mais formal sobre índices, podemos afirmar que:

> Um índice é uma estrutura de dados que organiza registros de dado em disco para otimizar determinados tipos de operações de recuperação. Um índice nos permite recuperar de forma eficiente todos os registros que satisfaçam as condições de pesquisa nos campos de chave de pesquisa do índice (RAMAKRISHNAN; GEHRKE, 2008, p. 228).

Assim, índices são empregados para otimizar buscas na tabela do banco de dados; contudo, a principal questão que fica é: quando devemos criar um índice? Segundo Price (2009), um índice precisa ser criado em uma coluna quando desejamos recuperar na consulta um pequeno número de linhas de uma grande tabela — nesse sentido, uma boa regra geral consiste em construir um índice quando uma consulta recuperar 10% ou menos do total de linhas de uma tabela.

Outro ponto importante na criação de um índice reside em qual coluna escolher para a indexação. Uma boa coluna candidata é aquela que contém a chave primária da tabela, já que esse tipo de chave geralmente é do tipo numérico, facilitando a indexação (BARBOSA; FREITAS, 2018). Outra é aquela que apresenta um valor exclusivo para cada uma das linhas da tabela, por exemplo, o número de matrícula de um aluno de uma faculdade. Já as colunas com pouca variedade de valores não são boas candidatas à indexação, como uma que armazene o sexo: feminino ou masculino. Ainda, destaca-se que índices podem ser criados sobre múltiplas colunas de uma tabela, e não necessariamente sobre somente uma coluna (PRICE, 2009).



**Fique atento**

Em um banco de dados relacional, a chave primária refere-se a uma coluna de uma tabela, ou uma combinação de colunas, em que os valores são diferentes em cada linha da tabela, ou seja, cada linha da tabela dispõe de uma chave primária com valores únicos. Boas práticas sugerem que a chave primária corresponda a um valor numérico para facilitar a criação de um índice sobre ela e que, ao definir uma chave primária em um banco de dados relacional, não se está definindo nenhum caminho de acesso aos dados, e sim apenas uma restrição de integridade, que é a unicidade dos valores (HEUSER, 2009).

Entre as principais vantagens na utilização de índices, está a consulta otimizada nas tabelas. Porém, eles apresentam algumas desvantagens, como a demora na inserção de novos registros na tabela, visto a necessidade de um tempo adicional para atualizar o índice na tabela (PRICE, 2009), além do fato de adicionarem sobrecarga ao sistema de banco de dados como um todo, o que mostra que devem ser usados com sensatez (POSTGRESQL..., 2020).

## Índices agrupados, primários e secundários

Diz-se que o índice sobre uma coluna de tabela é agrupado se a organização dos registros de dados for a mesma ou parecida com a ordenação das entradas de dados do índice; caso contrário, é definido como não agrupado. Já um índice sobre a coluna que contém a chave primária é chamado de ordenado, pois os registros estão ordenados pela chave primária (RAMAKRISHNAN; GEHRKE, 2008).

Um índice criado sobre a chave primária e outra coluna da tabela, por exemplo, também leva o nome de agrupado se o registro de dados estiver ordenado pela chave primária. Porém, na prática isso raramente acontece, já que ordenar os dados conforme sua atualização é muito custoso.

Um índice pode ser do tipo primário, criado sobre a coluna que contém a chave primária de uma tabela, ou secundário, quando contempla os demais tipos de índices. Entretanto, segundo Ramakrishnan e Gehrke (2008), essa definição ainda não está bem delimitada na literatura. Por exemplo, no SGBD PostgreSQL todos os índices são ditos secundários, porém a definição se diferencia: no PostgreSQL, um índice secundário é armazenado separadamente dos registros da tabela indexada (chamada de pilha) (POSTGRESQL..., 2020).

## Estrutura de dados de índice

Como vimos até agora, um índice compreende uma estrutura de dados que organiza registros de dados em disco. Uma das formas básicas utilizadas para organizar os registros de dados consiste em aplicar *hashing* sobre uma das colunas da tabela. Por exemplo, você pode indexar a coluna que armazena o nome dos funcionários em um registro de uma empresa, recuperando todos os registros do funcionário chamado João.

Na indexação que emprega *hash*, os registros são agrupados em estruturas chamadas *buckets*, que correspondem a uma página primária ou a páginas adicionais conectadas em uma cadeia. A função *hash* aplicada sobre a chave de pesquisa determina o *bucket* ao qual um registro pertencerá. A chave de pesquisa de um índice pode ser qualquer sequência de uma ou mais colunas de uma tabela e não precisa ser composta por registros únicos (RAMAKRISHNAN; GEHRKE, 2008).

Ao inserir um novo registro na tabela, ele é colocado no *bucket* apropriado. Pode-se alocar páginas extras para evitar que falte espaço para armazenamento. Quando você realiza uma consulta utilizando determinado valor de chave de pesquisa, aplica-se a função *hash* para encontrar o *bucket* ao qual o registro pertence e todas as páginas do *bucket* são examinadas. De acordo com o número de *buckets*, um índice do tipo *hash* pode recuperar a página primária em uma ou duas entradas/saídas no disco.

A Figura 1 mostra uma indexação utilizando *hash* sobre a coluna idade de uma tabela, caso em que os dados armazenados no primeiro arquivo de índice representam os registros dos dados reais. Logo, o índice não é armazenado em outra estrutura separada da tabela principal. A função *hash* `h` utilizada no exemplo é bastante simples, já que converte o valor da chave de pesquisa (`idade`) para sua representação binária e utiliza os dois *bits* menos significativos, como o identificador do *bucket*. Assim, ao aplicar a função `h` sobre a idade, você consegue identificar a página à qual o registro pertence.

Na Figura 1, também é mostrado um índice *hash* sobre a coluna `salário`, em que o `salário` e o `id` (`<salário, rid>`) estão armazenados no arquivo de índices. O componente `rid`, nesse caso, compreende a abreviação de `record id` — identificação do registro —, um ponteiro para um registro com valor de chave de pesquisa salário (RAMAKRISHNAN; GEHRKE, 2008).

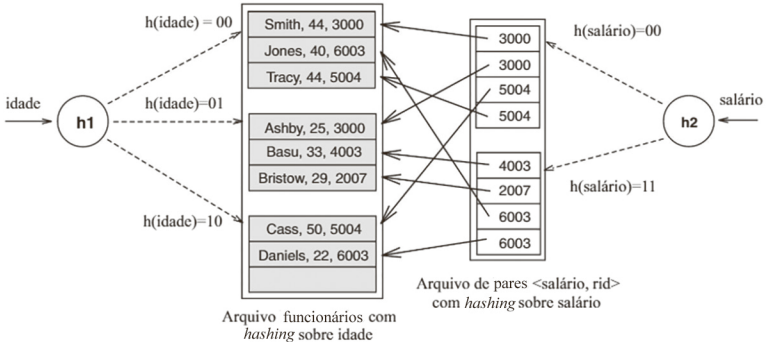



**Figura 1.** Arquivo organizado por índice com *hashing* sobre idade, com índice auxiliar sobre salário.

*Fonte:* Adaptada de Ramakrishnan e Gehrke (2008).

Outra maneira de organizar os registros de dados consiste em dispô-los de forma ordenada a partir do valor da chave de pesquisa em uma estrutura hierárquica no formato de árvore, que direciona as consultas para as páginas corretas (RAMAKRISHNAN; GEHRKE, 2008).

A Figura 2 apresenta os mesmos registros apresentados na Figura 1, porém indexados utilizando a árvore com a chave de pesquisa `idade`. Cada nó da árvore é uma página física, e a recuperação envolve entradas/saídas em disco. O menor nível da árvore chama-se folha (`L1`, `L2` e `L3`), contendo os registros dos dados.

A árvore permite localizar de maneira eficiente todos os registros de dados com valores de chave de pesquisa em um intervalo desejado. A pesquisa começa no nó raiz, que está no topo da árvore, e os conteúdos das páginas em níveis que não são folhas direcionam a consulta para a página folha correta a partir de ponteiros. O ponteiro do nó à direita de um valor de chave `k` aponta para uma subárvore, que dispõe apenas de registros de dados maiores ou iguais ao valor de `k`; já o ponteiro do nó à esquerda de um valor de chave `k` aponta para uma subárvore que contém apenas registros de dados menores que o valor de `k`. Se você desejar encontrar todas as entradas de dados com `33 < idade < 44`, verá direcionado a partir do nó raiz para a direita do nó `12`, para a direita do nó `19` e para a esquerda do nó `44` (RAMAKRISHNAN; GEHRKE, 2008).

Um tipo de árvore eficiente para utilizar em índices é a B-tree, que garante que todos os caminhos do nó raiz até o nó folha tenham o mesmo comprimento, tornando-se uma estrutura considerada balanceada pela altura. Esse tipo de índice é suportado por muitos SGBD, como o PostgreSQL (POSTGRESQL..., 2020).

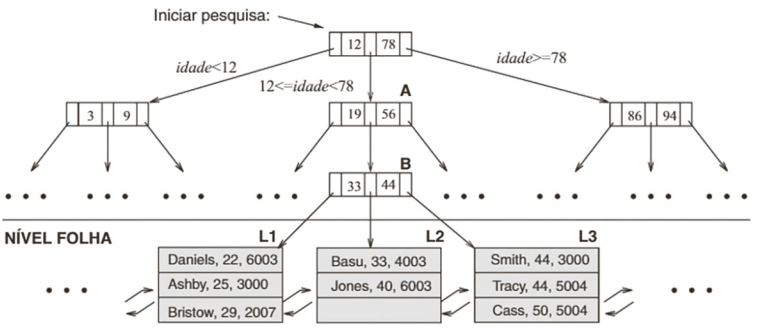



**Figura 2.** Índice estruturado como árvore (B-tree).
*Fonte:* Ramakrishnan e Gehrke (2008, p. 233).

# 2 Criação e exclusão de índices em banco de dados

O SGBD utilizado neste capítulo é o PostgreSQL, que, por padrão, cria índices do tipo B-tree, mas que também fornece vários outros tipos de índices, como Hash, GIN (Generalized Inverted Index), GiST (Generalized Search Tree), SP-GiST (Space-Partioned GiST) e BRIN (Block Range Index). Cada tipo de índice utiliza um algoritmo diferente para indexação, mais adequado em relação aos diferentes tipos de consultas (POSTGRESQL..., 2020; JUBA; VOLKOV, 2019). A partir de agora, você verá como criar cada um desses tipos e categorias de índices com o PostgreSQL, bem como excluir um índice.

## Tipos de índices

Para criar um índice com o PostgreSQL, basta utilizar o seguinte comando:

```
CREATE INDEX nome ON tabela (coluna);
```

Você deve indicar o nome do índice, a tabela em que deseja criá-lo e a coluna da tabela, podendo escolher qualquer nome para o índice. Uma dica é escolher algo que o ajude a lembrar mais tarde para que serve o índice. Como vimos, por padrão o PostgreSQL cria índices do tipo B-tree, que você pode utilizar caso pretenda realizar consultas sobre uma coluna da tabela envolvendo comparação utilizando um dos seguintes operadores: <, <=, =, >=, e >. Ainda, pode utilizar esse tipo de índice para recuperar dados classificados em ordem (POSTGRESQL..., 2020).



**Fique atento**

O PostgreSQL fornece diferentes operadores para utilizar em seleções, como (=) igual, (<>) diferente, (<) menor que, (>) maior que, (<=) menor ou igual, (>=) maior ou igual, (@>) contém o intervalo, (@>) contém o elemento, (<@) intervalo é contido por, (<@) elemento é contido por, (&&) sobreposição (tem pontos em comum), (<<) estritamente à esquerda de, (>>) estritamente à direita de, (&<) não se estende à direita de, (&>) não se estende à direita de, (– | –) é adjacente a, (+) união, (*) intersecção e (–) diferença (POSTGRESQL..., 2020).

Outro tipo de índice fornecido por esse SGBD é o índice *hash*, que consegue lidar apenas com comparações simples de igualdade (POSTGRESQL..., 2020). Você pode utilizar o seguinte comando para criar um índice *hash*:

```
CREATE INDEX nome ON tabela USING HASH (coluna);
```

Existem dois tipos de índices no PostgreSQL utilizados para acelerar as pesquisas em textos completos — o GIN e o GiST (POSTGRESQL..., 2020) —, que apresentam algumas diferenças. O tipo GiST suporta consultas utilizando os operadores <<, &<, &>, >>, <<|, &<|, |&>, |>>, >, <, ~= e &&; já o GIN suporta apenas consultas envolvendo <@, @>, = e &&. Além disso, o tipo GiST suporta indexação de vários tipos de dados geométricos bidimensionais. Você pode utilizar os seguintes comandos para criar esses tipos de índices:

```
CREATE INDEX nome ON tabela USING GIN (coluna);
CREATE INDEX nome ON tabela USING GIST (coluna);
```

No índice GIN, a `coluna` deve ser do tipo `tsvector`. E, no índice GiST, pode ser do tipo `tsvector` ou `tsquery`.

Outro tipo de índice similar ao GiST é o SP-GiST, empregado, por exemplo, para indexar dois pontos em um plano. Esse índice suporta consultas utilizando os operadores <<, >>, ~=, <@, <^ e >^. Você pode usar os seguintes comandos para criar esse tipo de índice:

```
CREATE INDEX nome ON tabela USING SPGIST (coluna);
```

O índice BRIN, projetado para lidar com tabelas muito extensas, em que determinadas colunas apresentam alguma correlação natural com sua localização física na tabela, apresenta um parâmetro chamado `pages_per_range`, que define o número de blocos de tabelas que compõem um intervalo de blocos para cada entrada de um índice BRIN — por padrão, é definido como `128` (POSTGRESQL..., 2020). Você pode criar um índice BRIN utilizando o seguinte comando:

```
CREATE INDEX nome ON tabela USING BRIN (coluna) WITH
(pages _ per _ range = 128);
```



**Saiba mais**

Digite "PostgreSQL BRIN índices como uma solução para Big Data" em seu mecanismo de busca preferido e confira o artigo de Rafael Santana e Ronan Silva sobre as vantagens do uso de índices BRIN para indexação de bancos com grandes volumes de dados (*big data*).

Os índices podem ser criados e excluídos das tabelas a qualquer momento. Para sua exclusão, você utiliza o seguinte comando, seguido do nome do índice a ser excluído (POSTGRESQL..., 2020):

```
DROP INDEX nome;
```

Além de criar índices em uma coluna, você pode escolher criar índices em múltiplas colunas de uma tabela (POSTGRESQL..., 2020), a partir do seguinte comando:

```
CREATE INDEX nome ON tabela (coluna1, coluna2, ...);
```

Atualmente, o PostgreSQL está na versão 12.2, a qual suporta apenas indexação de múltiplas colunas para os índices do tipo B-tree, GiST, GIN e BRIN com múltiplas colunas, podendo ser especificadas até 32 colunas.

Você também criar índices ordenados, suportados apenas por índices do tipo B-tree. Assim, é possível ajustar a ordem do índice incluindo as opções: `ASC` (ordem normal da tabela), `DESC` (ordem de trás para a frente), `NULL FIRST` (valores nulos primeiro) e/ou `NULLS LAST` (valores nulos no final). Por padrão, os índices são criados com os valores nulos por último (POSTGRESQL..., 2020). Esses índices podem ser criados a partir dos seguintes comandos:

```
CREATE INDEX nome ON tabela (coluna NULLS FIRST);
CREATE INDEX nome ON tabela (coluna DESC NULLS LAST);
```

Ainda, você consegue utilizar índices quando houver a necessidade de garantir a unicidade do valor de uma coluna, ou a unicidade dos valores combinados de várias colunas. Esse tipo de índice, que será único para cada linha da tabela, apenas pode ser do tipo B-tree. Para criá-lo, você utiliza o seguinte comando:

```
CREATE UNIQUE INDEX nome ON tabela (coluna [, ...]);
```

Você também pode criar índices utilizando funções, quando o índice será o resultado de uma função aplicada a uma ou mais colunas da tabela. Por exemplo, podemos criar um índice utilizando a função `lower` sobre uma coluna composta de caracteres, a partir do seguinte comando:

```
CREATE INDEX nome ON tabela (lower(coluna));
```



**Fique atento**

A função `lower` deve sempre ser aplicada em uma cadeia de caracteres, como o tipo `varchar`. Essa função é responsável por converter todas as letras da cadeia de caracteres em letras minúsculas. Por exemplo, aplicando a função `lower` sobre a cadeia de caracteres `'Joana Pereira da Costa'`, o resultado será `'joana pereira da costa'` (POSTGRESQL..., 2020).

Outro tipo de índice, chamado parcial,

> é um índice criado sobre um subconjunto de uma tabela; o subconjunto é definido por uma expressão condicional (chamada de predicado do índice parcial). O índice contém entradas apenas para as linhas da tabela que satisfazem o predicado (POSTGRESQL..., 2020, p. 400, tradução nossa).

Esse tipo é usado para evitar a indexação de valores comuns em uma tabela, reduzindo o tamanho do índice e acelerando as consultas e as operações de atualização. O predicado do índice parcial deve constar após o comando `WHERE`. Você pode criar um índice parcial para todos os registros em uma coluna que armazena idades, estabelecendo determinada faixa etária; por exemplo, sobre registros entre `50` e `80` anos de `idade`, deve-se utilizar o seguinte comando:

```
CREATE INDEX nome ON tabela (idade) WHERE (idade >= 50 AND
idade <= 80);
```

Ainda, podemos combinar categorias: por exemplo, para criar um índice parcial com unicidade de algum critério passível de teste, utilize o seguinte comando:

```
CREATE UNIQUE INDEX nome ON tabela (coluna) WHERE (condição
lógica);
```

Também é possível criar um índice parcial para excluir valores que não interessam. Por exemplo, para descartar todos os registros que são do sexo masculino, utilize o seguinte comando:

```
CREATE INDEX nome ON tabela (genero) WHERE (masculino is not
true);
```

No PostgreSQL, a criação de índices bloqueia inserções, exclusões e atualizações na tabela que está sendo indexada. O método `CONCURRENTLY` permite a construção de índices sem bloquear gravações na tabela, a partir do seguinte comando:

```
CREATE INDEX CONCURRENTLY nome ON tabela (coluna);
```

# 3 Aplicação de índices com SQL

A partir de agora, você aprenderá, por meio de exemplos de código PostgreSQL, como aplicar cada um dos tipos de índices vistos anteriormente.

## Exemplo 1: Hash e B-tree

Considere que você tem a seguinte tabela, cujos registros de dados estão ilustrados no Quadro 1:

```
CREATE TABLE exemplo1 (
    nome varchar,
    idade integer
);
```

**Quadro 1.** Registro de 6 pessoas em um banco de dados

| Nome | Idade |
|---|---|
| Bruna | 26 |
| Carla | 41 |
| Deiva | 78 |
| Fernando | 20 |
| Gilberto | 24 |
| Ilmo | 34 |

Você pode criar um índice do tipo *hash* sobre a coluna `idade` utilizando o seguinte comando:

```
CREATE INDEX id _ indice _ hash ON exemplo1 USING HASH (idade);
```

Ao empregar a chave de pesquisa sobre o índice `id_indice_hash`, você pode realizar uma consulta para achar todos os registros com idade igual a 20 anos, conforme o comando:

```
SELECT * FROM exemplo1 WHERE idade = 20;
```

Índices do tipo *hash* permitem apenas consultas de igualdade. Assim como os índices do tipo *hash*, os índices do tipo B-tree, além de possibilitarem consultas de igualdade, permitem consultas utilizando intervalos de busca. Por exemplo, ao tentar encontrar os registros com idade entre 30 e 80 anos, você pode criar um índice do tipo árvore. É possível criar um índice do tipo B-tree sobre o índice `id_indice_B` e realizar uma consulta utilizando os seguintes comandos:

```
CREATE INDEX id _ indice _ B ON exemplo1 (idade);
SELECT * FROM exemplo1 WHERE idade >= 30 AND idade <= 80;
```

A consulta sobre a tabela retorna os dados exibidos na Figura 3.

Data Output | Explain | Messages | Notifications

| | nome character varying | idade integer |
|---|---|---|
| 1 | Carla | 41 |
| 2 | Deiva | 78 |
| 3 | Ilmo | 34 |

**Figura 3.** Resultado da consulta utilizando intervalos.

## Exemplo 2: Dados geométricos

Considere que você tem a tabela seguir, cujos registros de dados são exibidos no Quadro 2.

```
CREATE TABLE exemplo2 (
    ponto point
);
```

**Quadro 2.** Registro com 11 pontos bidimensionais

| Ponto |
|---|
| (0,1) |
| (0,2) |
| (0,6) |
| (1,1) |
| (1,2) |
| (2,1) |
| (3,2) |
| (4,9) |
| (5,1) |
| (6,8) |
| (7,9) |

Existem dois tipos de índices possíveis para realizar consultas em uma tabela como essa: GiST e SP-GIST, a partir do seguinte comando sobre o campo ponto:

```
CREATE INDEX ponto_index_gist ON exemplo2 USING GIST (ponto);
CREATE INDEX ponto_index_spgist ON exemplo2 USING SPGIST
(ponto);
```

O tipo de índice escolhido dependerá do tipo de busca desejado — se você precisar encontrar os primeiros 6 pontos mais próximos do ponto `(0,1)`, pode utilizar o índice GiST para realizar a seguinte consulta para retornar 6 linhas da tabela:

```
SELECT * FROM exemplo2 ORDER BY ponto <-> point '(0,1)' LIMIT 6;
```

A consulta sobre a tabela retorna os dados exibidos na Figura 4.

Data Output | Explain | Messages | Notifications

| | ponto point |
|---|---|
| 1 | (0,1) |
| 2 | (0,2) |
| 3 | (1,1) |
| 4 | (1,2) |
| 5 | (2,1) |
| 6 | (3,2) |

**Figura 4.** Resultado da consulta em pontos.

## Exemplo 3: Múltiplas colunas

Considere que você tem a seguinte tabela:

```
CREATE TABLE exemplo3 (
    nome varchar,
    sobrenome varchar,
    maior float,
    menor float
);
```

Essa tabela tem o registro de dados de vendedores de uma loja, no qual são armazenados o `nome` e o `sobrenome` do vendedor, além da `maior` e da `menor` venda no mês, conforme o Quadro 3.

**Quadro 3.** Registro com 8 vendedores

| Nome | Sobrenome | Maior | Menor |
|---|---|---|---|
| Ana | Monteiro | R$ 5.000,00 | R$ 1.000,00 |
| Fernando | Fagundes | R$ 100,00 | R$ 90,00 |
| Flaviele | Oliveira | R$ 4.500,00 | R$ 100,00 |
| Gilberto | Moraes | R$ 1.400,00 | R$ 180,00 |
| Igor | Soares | R$ 3.300,00 | R$ 150,00 |
| Ilmo | Da Silva | R$ 3.400,00 | R$ 100,00 |
| João | Lima | R$ 2.400,00 | R$ 100,00 |
| Joana | Lopes | R$ 3.900,00 | R$ 450,00 |

Em uma tabela desse tipo, costuma-se realizar o seguinte tipo de consulta:

```
SELECT nome FROM exemplo3 WHERE maior = 100 AND menor = 9;
```

Você pode criar índices em múltiplas colunas, sobre as colunas `maior` e `menor`, otimizando as buscas com os valores dessas colunas como chave de pesquisa:

```
CREATE INDEX index _ multi ON exemplo3 (maior, menor);
```

Se, nesse caso, você utiliza a coluna `nome` mais vezes para realizar as buscas, pode criar um índice sobre o `nome` e o `sobrenome`, como um índice B-tree, ou um índice utilizando funções sobre o nome. Para isso, basta criar um índice em múltiplas colunas, empregando as colunas nome e sobrenome:

```
CREATE INDEX index _ f ON exemplo3 ((nome || ' ' || sobrenome));
```

É possível utilizar uma função para buscar um nome específico combinando o nome com o sobrenome, por exemplo, Joana Lopes:

```
SELECT * FROM exemplo3 WHERE ((nome || ' ' || sobrenome)) =
'Joana Lopes';
```

A consulta sobre a tabela retorna os dados apresentados na Figura 5.

Data Output | Explain | Messages | Notifications

| | nome character varying | sobrenome character varying | maior double precision | menor double precision |
|---|---|---|---|---|
| 1 | Joana | Lopes | 3900 | 450 |

**Figura 5.** Resultado da consulta combinando as colunas nome e sobrenome.

## Exemplo 4: Pedidos faturados

Se você tem uma tabela que contém pedidos faturados e não faturados, em que os pedidos não faturados ocupam um pequeno espaço na tabela e são as linhas mais acessadas, pode melhorar o desempenho criando um índice parcial apenas nas linhas não faturadas, a partir do seguinte comando:

```
CREATE INDEX nfaturado _ indice ON pedidos (num _ pedidos) WHERE
faturado is not true;
```

Uma possível consulta consiste em:

```
SELECT * FROM pedidos WHERE faturados is not true AND num _ pe-
didos < 10000;
```

Você também pode realizar consultas utilizando o índice com colunas que envolvem o número do pedido (`num_pedido`):

```
SELECT * FROM pedidos WHERE faturados is not true AND montante
> 5000.00;
```

Por se tratar de registros sobre vendas, pode ser interessante criar um índice que não bloqueie a operação de inserção de dados na tabela, com o comando CONCURRENTLY:

```
CREATE INDEX CONCURRENTLY nfaturado_indice ON pedidos
(num_pedidos)
WHERE faturado is not true;
```

Utilizando o mesmo exemplo da tabela de pedidos, suponha que ela contenha milhares de registros. O índice parcial já não será mais eficiente, visto que o intuito do uso desse tipo de índice consiste em retornar poucas linhas da tabela. Você pode utilizar o índice do tipo BRIN, mais indicado para tabelas muito grandes, utilizando o seguinte comando sobre o campo data:

```
CREATE INDEX * ON pedidos USING BRIN (data) WITH (pages_per_
range = 128);
```

## Exemplo 5: Collate

Considere a seguinte tabela:

```
CREATE TABLE exemplo4 (
    id integer,
    conteudo varchar COLLATE "x"
);
```

É possível também agrupar (COLLATE) índices no PostgreSQL. Um índice somente pode suportar um agrupamento por coluna de índice. Caso deseje realizar vários agrupamentos, serão necessários vários índices. O índice utiliza automaticamente o agrupamento da coluna subjacente. É possível utilizar o seguinte comando para criá-lo e realizar uma consulta:

```
CREATE INDEX collate_index ON exemplo4 (conteudo);
SELECT * FROM exemplo4 WHERE conteudo > 10;
```

Tal consulta pode usar o índice, já que a comparação empregará o agrupamento da coluna por padrão, embora não acelere consultas que envolvem outro agrupamento. Um índice adicional pode ser criado para suportar o agrupamento “y”:

```
CREATE INDEX collate _ index _ y ON exemplo (content COLLATE "y");
SELECT * FROM exemplo4 WHERE conteudo > 10 COLLATE "y";
```

## Exemplo 6: Busca em texto

Considere a seguinte tabela:

```
CREATE TABLE exemplo6 (
    id integer,
    titulo tsvector,
    corpo tsvector
);
```

Essa tabela tem o registro de dados de livros, com `id`, `titulo` e `corpo`, conforme mostrado no Quadro 4.

**Quadro 4.** Registro com 8 livros e suas informações principais

| id | titulo | corpo |
|---|---|---|
| 1 | The Lucky One | ... |
| 2 | The Last Song | ... |
| 3 | The Best of Me | ... |
| 4 | Dear John | ... |
| 5 | See Me | ... |
| 6 | Every Breath | ... |
| 7 | The Notebook | ... |
| 8 | A Walk to Remember | ... |

Se desejar realizar uma busca no texto completo do livro, você pode utilizar o índice GiST ou, então, o índice GIN para acelerar a consulta. Para criar um índice sobre as colunas `titulo` e `corpo`, podemos utilizar o seguinte comando:

```
CREATE INDEX indice_gin ON exemplo6 USING GIN((titulo || ' '
|| corpo));
```

A partir da criação desse índice, você pode realizar uma consulta do seguinte tipo:

```
SELECT * FROM exemplo6 WHERE (titulo || ' ' || corpo) = 'Dear
John ...';
```

A consulta sobre a tabela mostra os dados da Figura 6.

Data Output | Explain | Messages | Notifications

| | id integer | titulo tsvector | corpo tsvector |
|---|---|---|---|
| 1 | 4 | 'Dear' 'John' | '...' |

**Figura 6.** Resultado da consulta utilizando índices GIN.



## Saiba mais

Digite "Busca Textual no PostgreSQL é boa o suficiente" em seu mecanismo de busca preferido e confira o artigo de Marcelo Costa para aprender mais sobre o uso dos tipos de índices GIN e GiST para busca textual, com exemplos de suas criações.

## Verificar o uso de índices

É importante verificar quais índices são realmente utilizados em uma consulta real. O PostgreSQL oferece o comando `EXPLAIN` para realizar um exame do uso do índice para uma consulta individual, podendo ser utilizado como:

```
EXPLAIN SELECT * FROM tabela WHERE coluna = constante;
```

Além do comando `EXPLAIN`, deve-se executar o comando `ANALYZE`, que coleta estatísticas sobre a distribuição dos valores na tabela para estimar o número de linhas retornadas por uma consulta. Sem a utilização de parâmetros, o comando `ANALYZE` examina todas as tabelas no banco de dados atual. Já com a utilização de um parâmetro, o comando examina apenas a tabela desejada (POSTGRESQL..., 2020). Para utilizar `ANALYZE` em uma tabela específica, você pode utilizar o seguinte comando:

```
ANALYZE tabela;
```

Ainda, pode especificar uma coluna da tabela para aplicar o `ANALYZE`:

```
ANALYZE tabela (coluna);
```



### Referências

BARBOSA, F. F. M.; FREITAS, P. H. C. *Modelagem e desenvolvimento de banco de dados*. Porto Alegre: SAGAH, 2018. 188 p.

HEUSER, C. A. *Projeto de banco de dados*. 6. ed. Porto Alegre: Bookman, 2009. (Série Livros Didáticos Informática UFRGS, 4).

JUBA, S.; VOLKOV, A. *Learning PostgreSQL 11*: a beginner's guide to building high-performance PostgreSQL database solutions. 3. ed. Birmingham: Packt, 2019. 556 p.

POSTGRESQL 12.2 Documentation. [*S. l.*]: The PostgreSQL Global Development Group, 2020. 2661 p. Disponível em: https://www.postgresql.org/files/documentation/pdf/12/postgresql-12-A4.pdf. Acesso em: 15 abr. 2020.

PRICE, J. *Oracle database 11g SQL*: domine SQL e PL/SQL no banco de dados Oracle. Porto Alegre: Bookman, 2009. 684 p.

RAMAKRISHNAN, R..; GEHRKE, J. *Sistemas de gerenciamento de banco de dados*. 3. ed. Porto Alegre: AMGH; Bookman, 2008. 905 p.

## Leitura recomendada

SANTANA, R.; SILVA, R. PostgreSQL BRIN índices como uma solução para Big Data. *InfoQ*, [*S. l.*], 19 mar. 2019. Disponível em: https://www.infoq.com/br/articles/postgresql--bigdata/. Acesso em: 15 abr. 2020.



## Fique atento

Os *links* para *sites* da *web* fornecidos neste capítulo foram todos testados, e seu funcionamento foi comprovado no momento da publicação do material. No entanto, a rede é extremamente dinâmica; suas páginas estão constantemente mudando de local e conteúdo. Assim, os editores declaram não ter qualquer responsabilidade sobre qualidade, precisão ou integralidade das informações referidas em tais *links*.

Encerra aqui o trecho do livro disponibilizado para esta Unidade de Aprendizagem. Na Biblioteca Virtual da Instituição, você encontra a obra na íntegra.

Conteúdo:

SAGAH
SOLUÇÕES EDUCACIONAIS INTEGRADAS

# Dica do Professor: Índices em banco de dados

Os índices são muito úteis para organizar os registros de dados em um banco de dados. O arquivo que armazena os índices criados sobre uma tabela são, geralmente, armazenados em local diferente daquele que armazena a tabela indexada. Existem diferentes formas para a ordenação das entradas de dados no arquivo de índices. Uma forma de organizar as entradas de dados no arquivo de índices é ordenadamente em disco, de forma crescente e com os valores nulos no final. Além disso, existem parâmetros que podem ser utilizados para escolher a forma de ordenação.

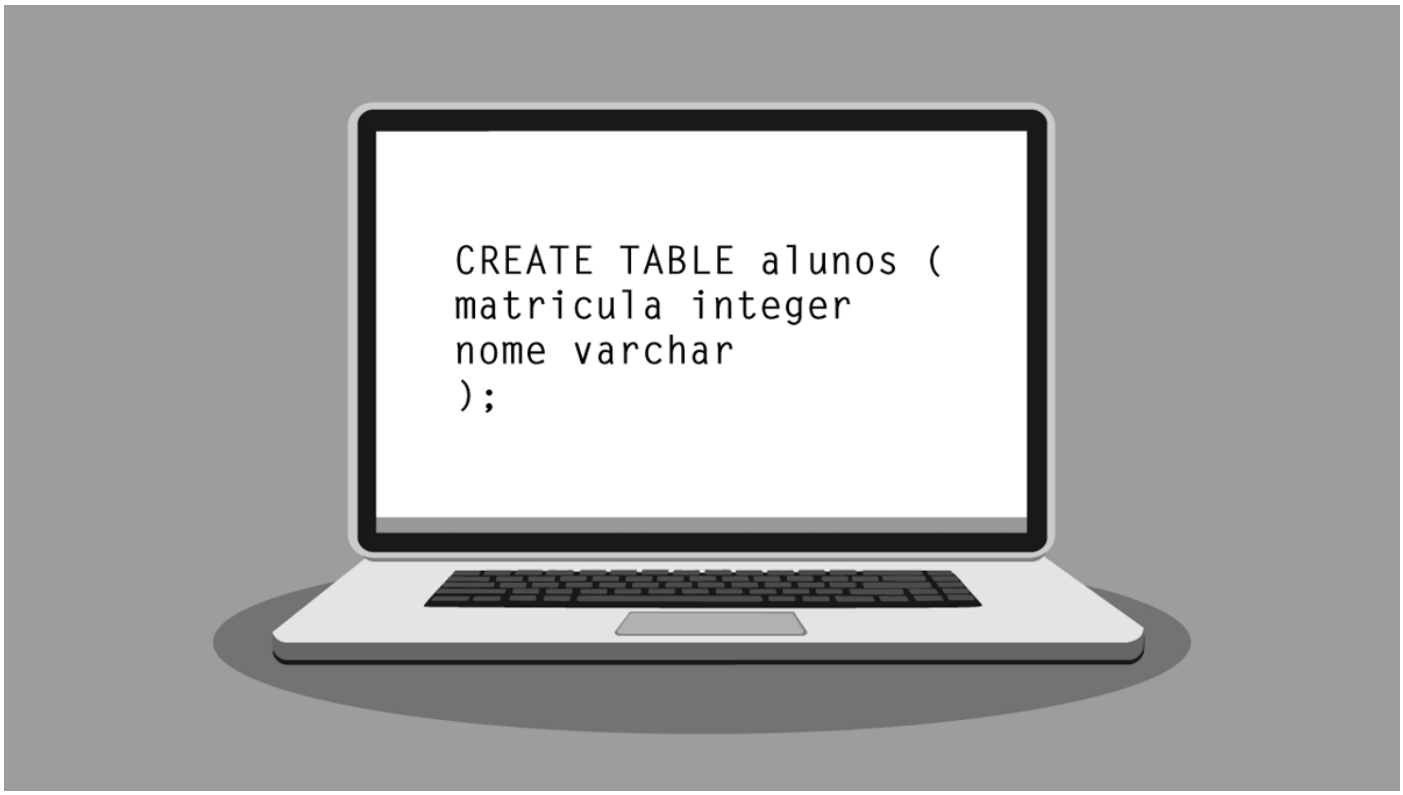
Na Dica do Professor, você verá quais as possíveis formas de ordenação das entradas de dados no arquivo de índices.



Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

# Atividade 2 (A2)

1) **Considere a seguinte tabela:**




**O administrador do banco de dados que trabalha em uma escola criou um índice do tipo *hash* sobre a coluna Matrícula com o objetivo de otimizar as consultas aos dados dos alunos.**

**Qual tipo de consulta é possível utilizando esse tipo de indexação?**

**[Revisor gramatical: não retirar o ponto e vírgula nas alternativas, pois fazem parte do código.]**

**A)** SELECT * FROM alunos WHERE matricula < 1211051017;

**B)** SELECT * FROM alunos WHERE matricula > 1010253654 AND matricula < 1211051017;

**C)** SELECT * FROM alunos WHERE matricula < 1211051017;

**D)** SELECT * FROM alunos WHERE matricula = 1211051017;

**E)** SELECT * FROM alunos WHERE upper(matricula) = 1211051017;

2) **Existem muitos tipos de índices disponíveis pelos diferentes Sistemas de Gerenciamento de Banco de Dados. Cada um deles funciona com determinado algoritmo e, por sua vez, cada**

**tipo de algoritmo é indicado para determinado tipo de consulta sobre determinado tipo de dado.**

**Quais são algoritmos básicos utilizados para a indexação de tabelas pela maioria dos Sistemas de Gerenciamento de Banco de Dados?**

A) *Hash* e BRIN.

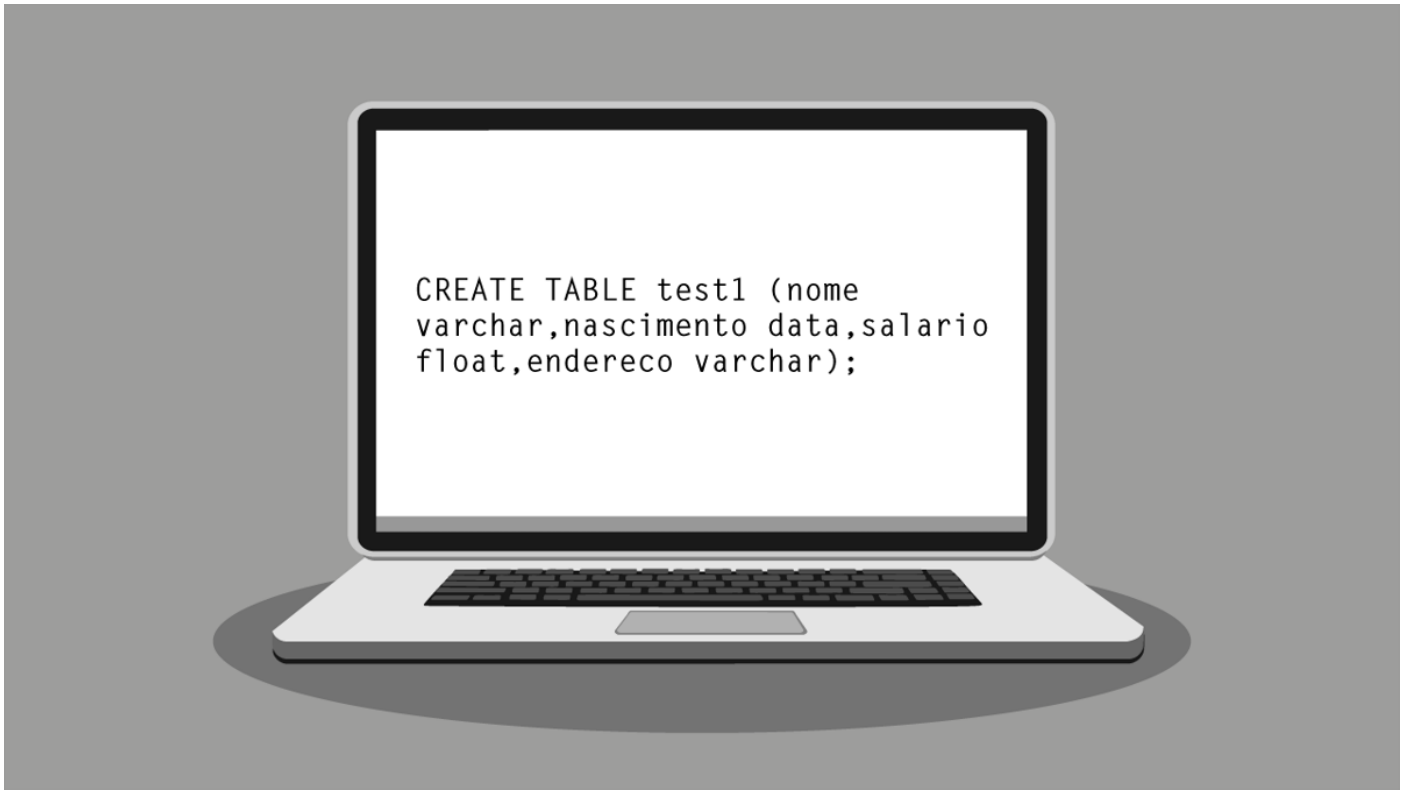
B) Árvore B e BRIN.

C) GIN e *hash*.

D) Árvore B e *hash*.

E) Árvore B e GIN.

3) **Considere a seguinte tabela:**




**O administrador do banco de dados de uma empresa multinacional quer otimizar as consultas nos registros de funcionários para obter informações sobre a idade dos funcionários e o valor das férias remuneradas deles. Ele deseja criar um índice principal e outro auxiliar.**

**Dentre as colunas da tabela, quais são as melhores candidatas para indexação?**

A) As colunas Nome e Nascimento são boas candidatas para a criação de índice.

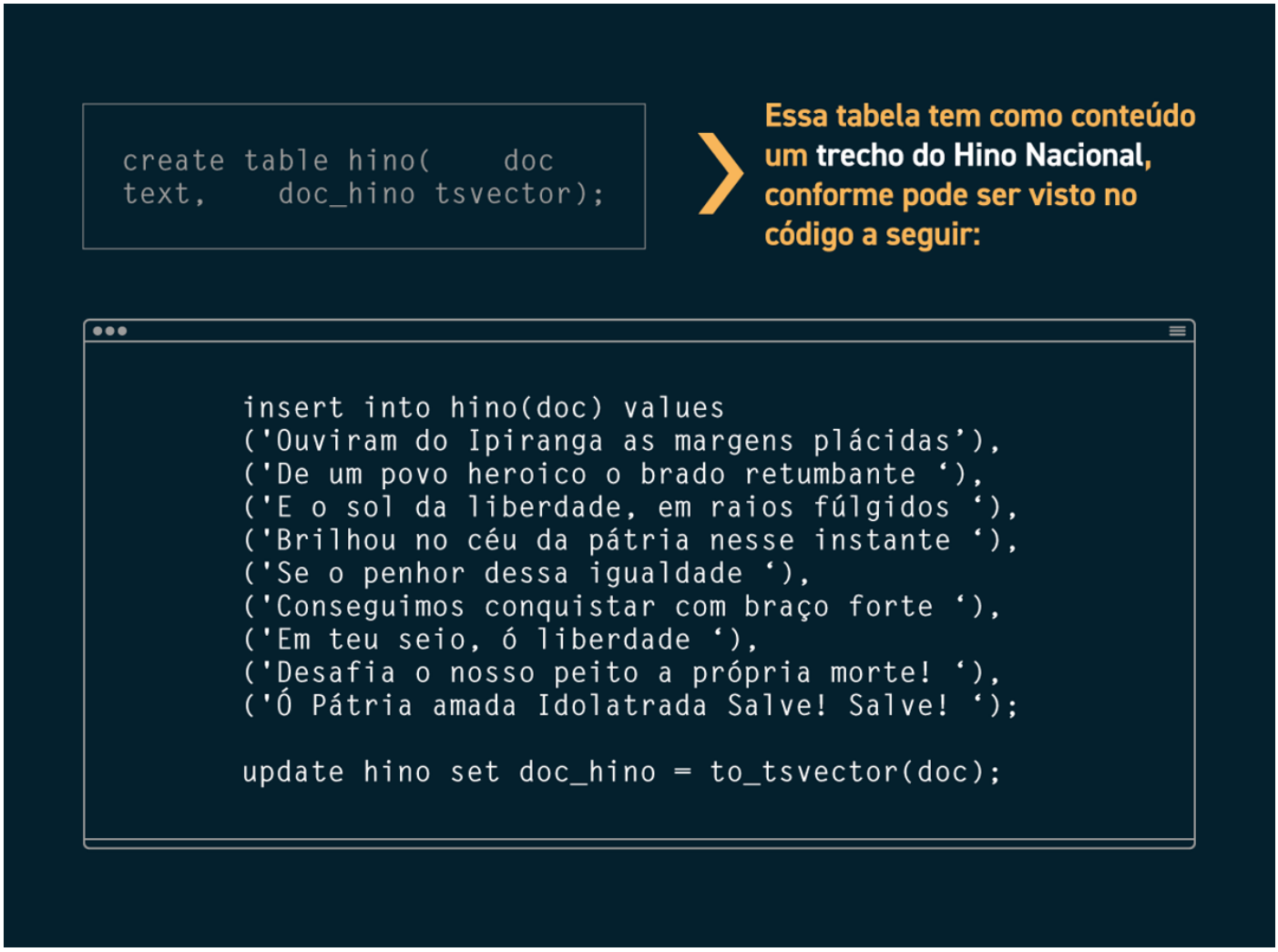
**B)** As colunas Nascimento e Salário são boas candidatas para a criação de índice.

**C)** As colunas Nome e Salário são boas candidatas para a criação de índice.

**D)** As colunas Nascimento e Endereco são boas candidatas para a criação de índice.

**E)** As colunas Nome e Endereço são boas candidatas para a criação de índice.

**4) Considere a seguinte tabela:**




**Para facilitar a consulta aos dados dessa tabela, é indicada a criação de índices para organizar os registros.**

**Assinale a alternativa que contém os tipos de índices podem ser criados sobre alguma coluna dessa tabela:**

**A)** GIN e BRIN.

**B)** GIN, BRIN e GiST.

**C)** BRIN e GiST.

**D)** BRIN e SP-GiST.

**E)** GIN e GiST.

**5)** **Índices são estruturas de dados adicionais utilizadas para organizar os registros de dados em disco com o objetivo de otimizar operações de recuperação de dados. No Sistema Gerenciador de Banco de Dados PostgreSQL, essas estruturas de dados não são armazenadas no mesmo local que os registros dos dados da tabela indexada.**

**No PostgreSQL, esse tipo de índice é chamado de:**

**A)** Índice Árvore B.

**B)** Índice *hash*.

**C)** Índice agrupado.

**D)** Índice primário.

**E)** Índice secundário.

# Saiba mais: Índices em banco de dados

Para ampliar o seu conhecimento a respeito desse assunto, veja abaixo as sugestões do professor:

## Breve introdução sobre índices

Confira este vídeo sobre índices, introduzindo os conceitos de Heap, Clustered e Nonclustered.

Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

## Vinte e cinco dicas e boas práticas de banco de dados para desenvolvedores

Confira esta postagem com algumas dicas e boas práticas para desenvolvedores de banco de dados, incluindo alternativas de quando e como utilizar índices.

Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

## Guia de otimização de consultas em PostgreSQL exemplificado com casos reais

Confira este vídeo com algumas dicas para otimizar consultas em banco de dados. Entre as dicas, estão o uso de índices e o uso do comando EXPLAIN.



Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

## Indexes In PostgreSQL

Confira uma série de artigos de um *blog* (em inglês) com exemplos do funcionamento da inserção e consulta utilizando índices.



Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.

## Armazenamento e manipulação de dados espaciais no PostgreSQL/PostGIS

Confira esta monografia, que apresenta o suporte de dados geográficos utilizando o índice Gis do PostgreSQL e o desenvolvimento de um estudo de caso.



Aponte a câmera para o código e acesse o link do conteúdo ou clique no código para acessar.